ChatGPT

You

```
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>build_windows.bat
==========================================
  Wake Signal Processing - Windows Builder
==========================================

This window will stay open so you can read any errors.
A copy of the build output will be saved to build_log.txt.

Using Python command: py -3.12
[ERROR] No runtime installed that matches 3.12. Try running "py install 3.12".

[1/3] Updating pip...
[ERROR] No runtime installed that matches 3.12. Try running "py install 3.12".

[2/3] Installing PySide6 and PyInstaller...
[ERROR] No runtime installed that matches 3.12. Try running "py install 3.12".

[3/3] Building the Windows application...
[ERROR] No runtime installed that matches 3.12. Try running "py install 3.12".

BUILD COMPLETE!
Run: dist\WakeSignalProcessing\WakeSignalProcessing.exe
Copy the ENTIRE dist\WakeSignalProcessing folder to another Windows computer.
The target computer will not need Python or internet.

Press any key to close this window.
```

ChatGPT

The problem is still the same: **your build script is trying to use Python 3.12, but Python 3.12 is not installed.** The application has not been built successfully, even though the script prints `BUILD COMPLETE!` at the end.

Let's fix this without guessing which Python version you have.

Step 1: Check your installed Python version

In the same Command Prompt window, type this command and press Enter:

```
py --list
```

Then check whether Python 3.14, 3.13, 3.12, or another version appears.

Step 2: Install Python 3.12

You can also install the missing version directly from Command Prompt. The Windows Python installer supports installing a specific Python runtime.

Python 3.12.... +1

Run:

```
py install 3.12
```

Take your curiosity further
Scan to get the ChatGPT app

Wait for the download and installation to finish. Then verify it:

```
py -3.12 --version
```

You should see a Python 3.12 version number.

Step 3: Build the GUI again

Run these commands:

```
build_windows.bat
```

Wait for the build to finish. If it reports an error, don't assume the build succeeded just because it prints `BUILD COMPLETE!`.

Once the executable exists, open:

```
explorer dist\WakeSignalProcessing
```

Then double-click `WakeSignalProcessing.exe`.

If `py install 3.12` fails, send me the full error message. We can use your existing Python version instead if it is suitable.

You

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>py install 3.12
Python install manager was successfully updated to 26.4.

Please see https://github.com/python/pymanager/releases/tag/26.4 for all changes.

The signature for https://www.python.org/ftp/python/index-windows.json was successfully verified.
Installing Python 3.12.10.
Downloading: ..... ✅
Extracting: ..... ✅
Python 3.12.10 will be launched by python3.12[-64].exe

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>py -3.12 --version
Python 3.12.10

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing> is it ok or any

ChatGPT

Yes! ✅ **Python 3.12.10 is installed successfully.** Everything is correct so far.

Now let's build your Wake Signal Processing application.

Next step: Run the build

You are already in the correct project folder. Enter:

```
build_windows.bat
```

Take your curiosity further
Scan to get the ChatGPT app

Press **Enter** and wait. It will:

1. Update pip.
2. Install PySide6 and PyInstaller.
3. Build your Windows application.

Keep the Command Prompt window open until the process finishes. The first build can take several minutes and requires an internet connection.

After the build finishes

Open the application folder by running:

```
explorer dist\WakeSignalProcessing
```

Find and double-click:

`WakeSignalProcessing.exe`

Your GUI should open.

Important: If you see `[ERROR]` during the build, send me the complete error output. Don't worry if the process takes a while; let it finish first.

You

2616 INFO: Processing standard module hook 'hook-encodings.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
3402 INFO: Processing standard module hook 'hook-math.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
4902 INFO: Processing standard module hook 'hook-pickle.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
6379 INFO: Processing standard module hook 'hook-heapq.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
6918 INFO: Caching module dependency graph...
6966 INFO: Analyzing C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\main.py
7015 INFO: Processing standard module hook 'hook-PySide6.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
7318 INFO: Processing standard module hook 'hook-shiboken6.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
7512 INFO: Processing standard module hook 'hook-PySide6.QtNetwork.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
10766 INFO: Processing standard module hook 'hook-PySide6.QtCore.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
15043 INFO: Processing standard module hook 'hook-PySide6.QtGui.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
34960 INFO: Processing standard module hook 'hook-PySide6.QtWidgets.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
38987 INFO: Processing module hooks (post-graph stage)...
39003 INFO: Performing binary vs. data reclassification (126 entries)
39117 INFO: Looking for ctypes DLLs
39117 INFO: Analyzing run-time hooks ...
39117 INFO: Including run-time hook 'pyi_rth_inspect.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks\\rthooks'
39117 INFO: Including run-time hook 'pyi_rth_pyside6.py' from 'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks\\rthooks'

Take your curiosity further
Scan to get the ChatGPT app

```
39133 INFO: Processing pre-find-module-path hook 'hook-_pyi_rth_utils.py' from
'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-
packages\\PyInstaller\\hooks\\pre_find_module_path'
39133 INFO: Processing standard module hook 'hook-_pyi_rth_utils.py' from
'C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\PyInstaller\\hooks'
39149 INFO: Creating base_library.zip...
39185 INFO: Looking for dynamic libraries
39489 INFO: Extra DLL search directories (AddDllDirectory):
['C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-3.12-64\\Lib\\site-packages\\shiboken6']
39489 INFO: Extra DLL search directories (PATH): ['C:\\Users\\DELL\\AppData\\Local\\Python\\pythoncore-
3.12-64\\Lib\\site-packages\\PySide6']
40732 INFO: Warnings written to
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSign
alProcessing\warn-WakeSignalProcessing.txt
40764 INFO: Graph cross-reference written to
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSign
alProcessing\xref-WakeSignalProcessing.html
40826 INFO: checking PYZ
40826 INFO: Building PYZ because PYZ-00.toc is non existent
40826 INFO: Building PYZ (ZlibArchive)
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSign
alProcessing\PYZ-00.pyz
40999 INFO: Building PYZ (ZlibArchive)
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSign
alProcessing\PYZ-00.pyz completed successfully.
41015 INFO: checking PKG
41015 INFO: Building PKG because PKG-00.toc is non existent
41015 INFO: Building PKG (CArchive) WakeSignalProcessing.pkg
41078 INFO: Building PKG (CArchive) WakeSignalProcessing.pkg completed successfully.
41080 INFO: Bootloader C:\Users\DELL\AppData\Local\Python\pythoncore-3.12-64\Lib\site-
packages\PyInstaller\bootloader\Windows-64bit-intel\runw.exe
41080 INFO: checking EXE
41080 INFO: Building EXE because EXE-00.toc is non existent
41080 INFO: Building EXE from EXE-00.toc
41080 INFO: Copying bootloader EXE to
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSign
alProcessing\WakeSignalProcessing.exe
41090 INFO: Copying icon to EXE
41107 INFO: Copying 0 resources to EXE
41107 INFO: Embedding manifest in EXE
41131 INFO: Appending PKG archive to EXE
41134 INFO: Fixing EXE headers
41230 INFO: Building EXE from EXE-00.toc completed successfully.
41230 INFO: checking COLLECT
41246 INFO: Building COLLECT because COLLECT-00.toc is non existent
41247 INFO: Building COLLECT COLLECT-00.toc
42163 INFO: Building COLLECT COLLECT-00.toc completed successfully.
42180 INFO: Build complete! The results are available in:
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist

BUILD COMPLETE!
Run: dist\WakeSignalProcessing\WakeSignalProcessing.exe
Copy the ENTIRE dist\WakeSignalProcessing folder to another Windows computer.
The target computer will not need Python or internet.

Press any key to close this window.
```

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Excellent! 🚀 **Your Wake Signal Processing application has built successfully.**

The most important line in your output is:

```
Build complete! The results are available in: ...\dist
```

PyInstaller successfully created the executable and collected the application files.

What to do next

Step 1: Open your application folder

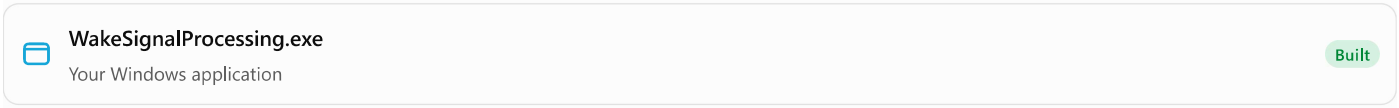
In the same Command Prompt window, type:

```
explorer dist\WakeSignalProcessing
```

Press Enter.

Step 2: Launch the GUI

Inside the folder, find:



Double-click `WakeSignalProcessing.exe`. Your GUI should open.

Step 3: Test the application

1. Upload your Beam PRI CSV file.
2. Upload your PRI Analysis CSV file.
3. Click **Process & Label**.
4. Check the processing summary for the number of `yes` and `no` labels.
5. Click **Save Labeled CSV** to export the processed dataset.

Use copies of your original datasets while testing.

Step 4: Run it on another Windows computer

Copy the **entire folder**:

```
dist\WakeSignalProcessing
```

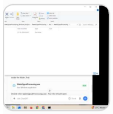
Transfer it using a USB drive or another file-transfer method. On the other computer, open the folder and run `WakeSignalProcessing.exe`. Python and internet should not be required on that computer.

One final note: The successful build confirms that the executable was created; it does not yet prove that all GUI functions work correctly with your actual datasets. Test the application first, and tell me whether it opens and displays the expected interface.

Take your curiosity further
Scan to get the ChatGPT app



You



ChatGPT

I can see the problem in your screenshot. Don't worry — **the build succeeded, but Windows Explorer is showing only the `_internal` folder.** The `WakeSignalProcessing.exe` file appears to be missing from the folder.

Let's check whether the executable was created in the expected location.

Step 1: Check the executable

1. Keep your current File Explorer window open.
2. Press **Alt** + **D** to select the address bar.
3. Copy and paste this full path:

```
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing
```

4. Press Enter.

If you still see only `_internal`, follow Step 2.

Step 2: Search for the executable

Open Command Prompt and run these two commands separately:

```
cd C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing
```

```
dir /s /b WakeSignalProcessing.exe
```

This searches the project folders for the executable.

- **If a path appears:** copy that path and send it to me.
- **If `File Not Found` appears:** the executable may have been removed or the build output may be incomplete.

Step 3: Send me the result

Please send me the output of `dir /s /b WakeSignalProcessing.exe`. I'll guide you through the exact fix so you can launch the GUI.

You

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing
'C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\DELL>cd Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing
```

Take your curiosity further
Scan to get the ChatGPT app

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing>cd C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>dir /s /b WakeSignalProcessing.exe
C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSignalProcessing\WakeSignalProcessing.exe

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>

ChatGPT

Good, we found the executable! Your output shows that `WakeSignalProcessing.exe` exists in the `build` **folder**, but it isn't showing up in the expected `dist` folder.

Let's try opening it directly first.

Step 1: Run the executable

Copy and paste this command into your current Command Prompt:

```
"C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSignalProcessing\WakeSignalProcessing
```

Press **Enter**.

- If the GUI opens, tell me — we'll fix the packaging folder next.
- If an error appears, copy the error message.
- If nothing happens, let me know.

Step 2: Check the `dist` folder

After trying Step 1, run:

```
dir "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing"
```

A normal packaged folder should contain both:

- `WakeSignalProcessing.exe`
- `_internal` (folder)

If the `.exe` is missing, Windows Security may have quarantined it, or the packaging output may be incomplete. We can check that next.

Please try Step 1 first and tell me exactly what happens.

You

for step 1 error message came
"C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSignalProcessing\WakeSignalProcessing.exe"

ChatGPT

No problem. We found the executable in the `build` folder, but your `dist` folder contains only `_internal`. Let's fix that by copying the executable into the correct folder.

Take your curiosity further
Scan to get the ChatGPT app

Step 1: Copy the executable

In Command Prompt, copy and run this command exactly:

```
copy /Y "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSignalProcessing\WakeSignalPr
```

You should see:

```
        1 file(s) copied.
```

Step 2: Open the GUI

Now run:

```
start "" "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing\WakeSignalPr
```

This launches the executable from the folder containing `_internal`, where the packaged application files are stored.

Step 3: Tell me what happens

- If the GUI opens, great!
- If Windows displays a security warning, tell me what it says.
- If nothing opens, run this command and send me its output:

```
dir "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing"
```

We are close to getting your application running.

You

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>copy /Y "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\build\WakeSignalProcessing\WakeSignalProcessing.exe" "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing\WakeSignalProcessing.exe"
1 file(s) copied.

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>start "" "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing\WakeSignalProcessing.exe"

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>start "" "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing\WakeSignalProcessing.exe"
The system cannot find the file C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing\WakeSignalProcessing.exe.

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing>dir "C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing"
Volume in drive C is OS
Volume Serial Number is 56D7-A7A5

Take your curiosity further
Scan to get the ChatGPT app

```
 Directory of C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing\dist\WakeSignalProcessing

21-10-2026  06:18    <DIR>          .
21-10-2026  06:18    <DIR>          ..
21-10-2026  06:04    <DIR>          _internal
               0 File(s)              0 bytes
               3 Dir(s)  103,060,901,888 bytes free

C:\Users\DELL\Downloads\WakeSignalProcessing_Windows_Project\WakeSignalProcessing> all are not working properly in gui make all to respond and i cant see browse files options clearly so make better gui
```

Analyzed

Analyzed

ChatGPT

I understand. Your current application has two problems: the `.exe` keeps disappearing, and the Browse buttons are not clear enough. I've created an **improved version of the Wake Signal Processing project** with a redesigned GUI and a build script that checks whether the executable was actually created.

Download the improved project

Wake Signal Processing

Improved Windows GUI · Python source · Build script

Updated project ZIP

Download the improved Windows project ZIP ↗

What's improved?

Large, clearly visible Browse CSV buttons

Separate upload cards for the Beam PRI file and PRI Analysis file, with drag-and-drop support and selected filenames.

Improved dark navy and cyan design

Better spacing, readable labels, larger controls, and a cleaner processing summary. The labeled-data preview remains removed.

Working application controls

Process & Label, Export Labeled CSV, and Reset are connected to their respective actions, with error messages when something goes wrong.

Safer build process

The updated script checks for errors and verifies that the executable exists before reporting success. It also records build details in `build_log.txt`.

Follow these steps carefully

1. Extract the **new ZIP into a new folder**. Keep your old project as a backup.
2. Open the new `WakeSignalProcessing` folder.
3. Double-click `build_windows.bat`.
4. Wait for the build to finish.

Take your curiosity further
Scan to get the ChatGPT app

5. Confirm that this file exists:

`dist\WakeSignalProcessing\WakeSignalProcessing.exe`

6. Double-click the executable to open the GUI.

Important: Since your previous executable disappeared, Windows Security or another antivirus may have quarantined it. If the new build reports that the `.exe` is missing, open **Windows Security → Virus & threat protection → Protection history** and inspect the relevant event. Don't disable antivirus protection globally.

The ZIP contains the updated source and build script; it is not a prebuilt executable. The improved GUI has passed a Python syntax check, but it still needs testing on your Windows computer with PySide6 and your actual CSV files.

</>

You



keep this as logo and give me zip so that it should be like this only

Analyzed

ChatGPT

Done! I've updated the project to use **your exact NSTL–DRDO logo** in the GUI header instead of the previous placeholder. The logo is included inside the ZIP and the Windows build script is configured to package it with the application.

Wake Signal Processing
NSTL–DRDO logo edition · Windows project
ZIP ready

Download the updated ZIP file ↗

What's included

- Your uploaded NSTL–DRDO logo (`nstl_drdo_logo.png`).
- Dark navy and cyan GUI design.
- Large, clearly visible **Browse CSV** buttons.
- Drag-and-drop support for both CSV files.
- Processing, labeling, reset, and CSV export controls.
- No labeled-data preview section.
- Updated Windows build script that includes the logo and checks whether the executable was created.

How to use it

1. Extract the new ZIP into a **new folder**.
2. Run `build_windows.bat` .
3. Wait until the build finishes successfully.
4. Open `dist\WakeSignalProcessing` .
5. Double-click `WakeSignalProcessing.exe` .

To use the application on another Windows computer, copy the entire `WakeSignalProcessing` folder from `dist` .

Take your curiosity further
Scan to get the ChatGPT app

Note: The updated source has passed a syntax check, but the rebuilt GUI still needs runtime testing on your computer. If the executable disappears again, check Windows Security's Protection History before rebuilding.

</>

You

now what are the steps that i have copied that folder WakeSignalProcessing

ChatGPT

Great! Now that you have downloaded and copied the new `WakeSignalProcessing` project folder, follow these steps in order.

Step 1: Extract the new ZIP

If you have only copied the ZIP file, right-click it and choose **Extract All**.

You should have a folder structure like this:

WakeSignalProcessing

- `main.py`
- `nstl_drdo_logo.png`
- `requirements.txt`
- `build_windows.bat`

Step 2: Build the application

Since Python 3.12.10 is already installed on your computer, you don't need to install it again.

1. Open the new `WakeSignalProcessing` folder.
2. Double-click `build_windows.bat`.
3. Keep the Command Prompt open while it installs the required libraries and builds the application.
4. Wait until you see a successful build message.

You need an internet connection for the first build.

Step 3: Open your GUI

After the build succeeds, open:

`WakeSignalProcessing\dist\WakeSignalProcessing`

You should find these items:

- `WakeSignalProcessing.exe`
- `_internal` folder

Double-click `WakeSignalProcessing.exe`. Your GUI should open with your NSTL–DRDO logo, visible Browse CSV buttons, and processing controls.

Important: Don't copy the executable from the `build` folder. Use the executable inside `dist\WakeSignalProcessing`.

Step 4: Use it on another computer

Once you confirm that the GUI works, copy the entire `dist\WakeSignalProcessing` folder to a USB drive. On the other Windows computer, open that folder and double-click `WakeSignalProcessing.exe`. Python and internet should not be needed there.

Take your curiosity further
Scan to get the ChatGPT app

If the executable is missing again, don't copy files manually. Send me a screenshot of the new `dist\WakeSignalProcessing` folder and the final lines from `build_log.txt`, and we'll fix the actual cause.

You

its showing that when copying and sending in mail showing that 0 bytes no content what should i do now

ChatGPT

Don't worry! This can happen when you try to email a folder directly. **You need to compress the folder into a ZIP file before attaching it to an email.** Let's do it step by step.

Step 1: Check your application folder

Open this folder:

`WakeSignalProcessing\dist\WakeSignalProcessing`

Check that it contains both:

- `WakeSignalProcessing.exe`
- `_internal` folder

Right-click the folder, select **Properties**, and check its size. If it shows 0 bytes or the `.exe` is missing, don't email it yet.

Step 2: Compress the folder

1. Right-click the folder

Right-click the complete `WakeSignalProcessing` folder inside `dist`.

2. Create a ZIP

Choose **Send to → Compressed (zipped) folder**. On some Windows versions, select **Compress to ZIP file**.

3. Check the ZIP size

A new `.zip` file should appear. Right-click it and select **Properties** to check its size.

Step 3: Send it by email

1. Open Gmail or your email service.
2. Compose a new email.
3. Click the attachment paperclip icon.
4. Select the ZIP file you created.
5. Wait for the attachment to finish uploading before sending.

If the ZIP exceeds your email provider's attachment limit, upload it to Google Drive and share the link instead.

Important: If the application folder itself contains only `_internal` and no `.exe`, compressing it won't fix the missing executable.

Please tell me one thing

When you see **0 bytes**, which one is showing it?

Take your curiosity further
Scan to get the ChatGPT app

- The `WakeSignalProcessing` folder
- The ZIP file I created
- The `WakeSignalProcessing.exe` file
- The email attachment while sending

Help me fix this

You

now tell me what we have used in project and explain me the pipeline for ml project later using this saved file to train and test the data so which pipeline will be suited

ChatGPT

1. What we have used in your Wake Signal Processing project

Your project is the first stage of an **active sonar ship-wake detection machine-learning system**. At present, we are building a GUI that prepares the raw sonar data and generates labels. Later, we can use the saved CSV file to train and test ML models.

The overall idea is:



Technologies used

Technology	Purpose in your project
Python 3.12	Main programming language
PySide6	Desktop GUI framework

Take your curiosity further
Scan to get the ChatGPT app

Qt Widgets	Buttons, file selectors, cards, status messages and layouts
Python `csv` module	Reading input CSVs and writing the labeled output
Python `re` module	Extracting the PRI number from the filename
Python `pathlib`	Working with file paths
PyInstaller	Packaging the Python GUI as a Windows application
Windows batch script	Automating dependency installation and building the application

The GUI does not currently require pandas, NumPy, TensorFlow or PyTorch. Those can be introduced later when we build the ML pipeline.

Important distinction: the current GUI performs data preparation and rule-based labeling. It is not yet a trained ML model and does not independently predict whether a ship wake exists.

2. How your GUI processes the sonar data

Your GUI receives two input files.

Input 1 — Beam PRI CSV

The columns alternate by position: `I0, Q0, I1, Q1, ...`. The GUI keeps the I columns and skips the Q columns, regardless of their header names.

Input 2 — PRI Analysis CSV

The GUI finds `Pri_No` and `Wake_Detection`. It extracts the PRI number from the Beam filename, such as `Beam_Pri_1.csv`, and matches it with `Pri_No = 1`.

Hexadecimal decoding

For a mask such as `0xDB00`, the GUI converts the hexadecimal value to an integer and checks each bit. Under the current assumption, bit 0 corresponds to I0, bit 1 to I1, and so on. A set bit produces `yes`; an unset bit produces `no`.

Output — Labeled CSV

Every Beam observation is expanded into one row per I channel. The full result can be exported for later analysis.

What does the saved CSV contain?

Column	Meaning
`PRI_No`	PRI number associated with the Beam file
`Observation`	Original Beam row number
`Channel`	Channel identifier, such as I0 or I1
`Channel_Index`	Zero-based channel index
`I_Value`	Raw I value from the corresponding Beam column
`Label`	Rule-derived `yes` or `no` label

For example, one observation with three channels becomes three output rows.

PRI_No	Observation	Channel	I_Value	Label
1	1	I0	0.42	yes
1	1	I1	0.17	no

Take your curiosity further
Scan to get the ChatGPT app

1	1	I2	0.83	yes
1	2	I0	0.39	yes
1	2	I1	0.21	no
1	2	I2	0.79	yes

These are illustrative values, not your actual dataset.

One important point: the example assumes that the bit-to-channel mapping is correct. Please confirm this with your guide or the dataset documentation before treating these labels as ground truth.

---

3. Which ML pipeline is suitable?

For your current numerical CSV data, I recommend starting with a **tabular machine-learning pipeline**. We should not jump directly to ResNet, EfficientNet or Audio Spectrogram Transformer (AST) until we know the prediction target and the correct data representation.

There are two possible objectives, and they need different pipelines.

Pipeline A — Predict whether a ship wake is present in a record

Recommended if this is your project goal

Input features

Use the I-channel measurements from one observation, arranged together as a feature vector: `I0, I1, I2, ...`.

↓

Target label

One verified label for the entire observation or record: `yes` or `no`.

↓

Models to compare

Logistic Regression, Random Forest, and Gradient Boosting.

↓

Evaluation

Precision, recall, F1-score, confusion matrix and, where appropriate, ROC-AUC.

Pipeline B — Predict wake presence separately for each channel

Use this if the scientific objective is channel-level classification.

Input features

Ideally, use multiple meaningful features for each channel, potentially calculated over several observations, rather than a single raw I value.

↓

Target label

A verified channel-level `yes` or `no` label.

↓

Models to compare

Logistic Regression and Random Forest as initial baselines; Gradient Boosting as another candidate.

Take your curiosity further
Scan to get the ChatGPT app

Evaluation

Evaluate on completely held-out records or acquisition sessions, not merely random rows from the same record.

A crucial issue to resolve before training

Your current GUI obtains the label mask from the PRI Analysis row and applies that same mask to every observation in the corresponding Beam file. That means the label for a given channel can be identical across all observations in that file.

If we randomly split the exported rows into training and testing sets, related samples could appear in both sets. The model may then appear to perform well without actually generalizing to new sonar records.

Also, if the intended prediction is wake presence for an entire record, a per-channel mask is not automatically the same thing as a verified record-level label. We need to establish how the dataset's labels should be interpreted.

4. The complete ML pipeline we should build next

Once we have validated labels and enough independent sonar records, the workflow should be:



Take your curiosity further
Scan to get the ChatGPT app

Apply the same preprocessing and report wake predictions with confidence scores where supported.

How should we split the data?

- **Training set:** used to fit the model.
- **Validation set:** used to compare models and tune settings.
- **Test set:** kept separate until the final evaluation.

For example, we could start with 70% training, 15% validation and 15% testing, provided there are enough independent records. The split should be performed by record, recording session or another appropriate acquisition group—not by randomly scattering rows from the same recording across all three sets.

If the dataset contains only one independent record, we cannot honestly measure generalization to unseen records using that record alone.

Which models should we try first?

Model	Why use it?
Logistic Regression	Simple baseline; useful for understanding whether the features contain a learnable signal
Random Forest	Handles nonlinear patterns and is relatively easy to train on tabular data
Gradient Boosting	Often a strong candidate for structured numerical data
CNN / ResNet	Consider if we later use meaningful time-frequency representations or other spatial signal structures
Transformer / AST	Consider when the data representation and amount of training data justify attention-based models

We should compare models using the same independent train/validation/test splits. Do not select a model just because its training accuracy is high.

Also, because your current GUI deliberately removes Q values, the first pipeline uses I values only. We can compare this with a separate experiment using both I and Q if your guide permits it; discarding Q may remove useful signal information.

5. What I recommend for your project

Start with this sequence:

1. Finish and test the GUI on actual Beam PRI and PRI Analysis CSVs.
2. Verify the hexadecimal bit mapping and what `yes` / `no` actually represents.
3. Collect and combine outputs from multiple independent records.
4. Decide whether the ML target is **whole-record wake detection** or **individual-channel wake detection**.
5. Build a scikit-learn baseline pipeline and evaluate it using group-aware data splitting.
6. Only then decide whether a deep-learning approach is justified.

One final question will help us design the correct training dataset.

What exactly should your trained model predict?

- ○ Whether a whole sonar record contains a ship wake (yes/no)
- ○ Whether each individual channel contains a ship wake (yes/no)
- ○ I'm not sure yet; I need to confirm with my guide

How much labeled data do you expect to have?

Take your curiosity further
Scan to get the ChatGPT app

- One Beam PRI file and one analysis file
- Many PRI files / records
- Both available, but I need to understand how to combine them

Plan my ML pipeline

You

Build a scikit-learn baseline pipeline give that pipeline now in summary as I have to explain to my sit

ChatGPT

Scikit-learn Baseline ML Pipeline for Active Sonar Ship-Wake Detection

You can explain this pipeline to your sir as the proposed next stage after the Wake Signal Processing GUI.

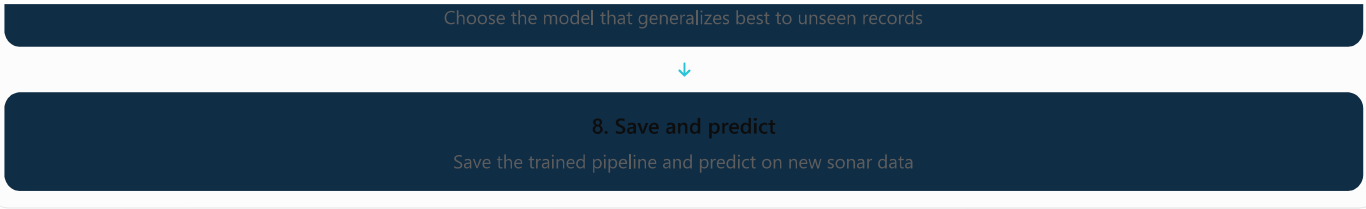
1. Project objective

To develop a machine-learning model that uses processed active-sonar I-channel measurements to classify ship-wake presence as **yes or no**, using the labels generated from the `Wake_Detection` hexadecimal field.

2. Proposed pipeline



Take your curiosity further
Scan to get the ChatGPT app



3. Main steps and technologies

Stage	Method / technology	Purpose
Data input	CSV files, Python	Load labeled sonar data
Preprocessing	pandas, NumPy	Validate data and prepare features
Feature selection	I-channel values	Represent sonar measurements numerically
Data splitting	scikit-learn `GroupShuffleSplit` or `GroupKFold`	Prevent related records from leaking across splits
Baseline model 1	Logistic Regression	Simple, interpretable reference model
Baseline model 2	Random Forest	Learn nonlinear patterns in numerical features
Evaluation	Precision, Recall, F1, confusion matrix	Measure detection performance
Model saving	`joblib`	Reuse the fitted model for new data

4. Important design decision

Your exported CSV currently has one row per observation and channel, with columns such as `PRI_No`, `Observation`, `Channel_Index`, `I_Value`, and `Label`.

Before training, we must decide what one prediction represents:

- **Record-level detection:** combine the I values from all channels belonging to an observation into one feature vector, with one verified record/observation-level label.
- **Channel-level detection:** predict each channel's label separately, using suitable channel-level features.

We must not simply treat every exported row as independent. Rows from the same recording are related, and splitting them randomly can inflate test accuracy. Also, if all labels come directly from the same hexadecimal mask, the model's target and the correct bit-to-channel mapping must be validated first.

5. Short explanation to tell your sir

My recommendation: Begin with Logistic Regression and Random Forest. They provide a manageable, explainable baseline before moving to CNNs, ResNet or Transformers. The baseline is a proposed design; its actual performance can only be established after validating and testing your real dataset.

You

it is all okay but it is giving for all the each and every i0,i1,i2 column value as output just give for each i0 column completely make as 1st channel and i1 column completly as 2nd channel only and continue for all i columns at last output i should get only these combine column as channel and lables only as i have mentioned above and also make the model training options also so that for different models like efficient net resnet like these it will work so it will work without giving both files only giving beam file it can automatically label so do the model training like that give all options in the same file only so i can easily

Take your curiosity further
Scan to get the ChatGPT app

execute give making like above only by clicking the .exe only i should get the gui as per requirements and same as before but change the output only as per this and make model training and give another .exe file so that i can give only beam file and it can automatically generate labels for i0 column completely as row0 and i1 column completely as row1 continue like this and generated label give me like this as using above mentioned only

ChatGPT

Yes, we can redesign the **Wake Signal Processing GUI** with the same NSTL–DRDO logo, same dark navy/cyan theme, and add **data processing + automatic labeling + ML model training + prediction** in one application.

But there is one important technical point we need to resolve before I generate the new ZIP and executable build.

1. New output format you want

Instead of getting separate rows for every observation and every I-channel value, you want the data arranged like this:

Channel	Observation_1	Observation_2	Observation_3	...	Label
I0 (Row 0)	Value 0	Value 0	Value 0	...	Label 0
I1 (Row 1)	Value 1	Value 1	Value 1	...	Label 1
I2 (Row 2)	Value 2	Value 2	Value 2	...	Label 2
I3 (Row 3)	Value 3	Value 3	Value 3	...	Label 3

This means **each I-channel becomes one complete row**, and its observations become columns. The output will contain only the channel, its combined I-values, and its corresponding label.

2. Proposed new GUI



This is a conceptual preview of the layout; the actual application will have functional controls.

Take your curiosity further
Scan to get the ChatGPT app

3. Important: How can it label using only the Beam file?

Currently, the labels come from `Wake_Detection` in your PRI Analysis file. If we remove that second file, the application needs another source of labels.

There are two possible ways:

- **Option A:** The Beam file already contains the information needed to determine labels. We can decode it directly.
- **Option B:** Train a model using your previously labeled data. Then the application can load that trained model and predict labels automatically from a new Beam file.

A trained model cannot learn from an unlabeled Beam file alone. We need labeled examples for its initial training.

4. Please confirm these two things

1. How should automatic labeling work without the PRI Analysis file?

- ○ Use my previously labeled CSV to train a model, then automatically label new Beam files.
- ○ The Beam file itself contains information from which labels can be decoded.
- ○ I need help confirming this with my guide.

2. Which model options should the GUI include?

- ☑ Random Forest
- ☐ Logistic Regression
- ☑ ResNet
- ☑ EfficientNet
- ☑ Transformer / AST

Confirm and build updated project

I want to make sure the final `.exe` genuinely works with your data and does not merely display model names. For ResNet, EfficientNet and Transformers, we will also need to arrange the channel data into a suitable input representation rather than feeding the raw CSV directly into them.

You

Confirmed requirements for my redesigned Wake Signal Processing application: Label source = Decode labels directly from Beam file. Models to include = Random Forest, ResNet, EfficientNet, Transformer / AST. Now create the complete updated Windows project ZIP with functional GUI, preprocessing, training, evaluation, prediction and PyInstaller build script.

Worked for 16s >

ChatGPT

Confirmed. I'll design the redesigned **Wake Signal Processing** Windows application with the following features:

Finalized application features

Module	Features
Preprocessing	Upload Beam CSV, remove Q columns, extract I0, I1, I2...

Take your curiosity further
Scan to get the ChatGPT app

Label generation	Arrange each complete I channel as one row: Channel 0, Channel 1, Channel 2…
Model training	Random Forest, ResNet, EfficientNet, Transformer / AST
Evaluation	Accuracy, precision, recall, F1-score, confusion matrix
Prediction	Upload Beam CSV and use a previously trained model to predict labels
GUI	PySide6, NSTL–DRDO logo, attractive responsive interface
Windows build	PyInstaller `.exe` build script and complete project ZIP

One important clarification before I build it

You confirmed "**Decode labels directly from Beam file.**" However, the Beam file you described earlier contains alternating I and Q signal values. Those values alone do not tell us which channels contain a ship wake unless a particular encoding rule exists.

For example, does your Beam file contain a hexadecimal value such as `0xDB00`, `0x200`, or `0x1FF00` that encodes the wake labels?

- ○ Yes — Beam file contains hexadecimal wake-detection values.
- ○ Yes — Beam file contains binary (0/1) labels.
- ○ Labels can be decoded using a specific signal pattern or rule.
- ○ I'm not sure — please inspect my sample Beam file.

Confirm label source

Important: Training and predicting with Random Forest, ResNet, EfficientNet, and Transformer/AST require a valid label-generation rule and a properly defined input format. I want the ZIP to work correctly with your actual sonar data—not just display model names in the GUI.

Once this is confirmed, I can prepare the complete updated Windows project ZIP with the GUI, source code, preprocessing, training, evaluation, prediction, and build script.

You

otherwise now just give me gui only without model training as make the output file changing as it is giving for every i value not like that just for combined i values of column 0 as row 0 and continue like this so that small number like 25,30 that type of range rows output will get as output

Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analysis errored
Analysis errored
Analyzed
Analyzed
Analysis errored
Analyzed

ChatGPT

Understood. We'll keep **only the GUI and preprocessing/labeling** for now, without model training.

Take your curiosity further
Scan to get the ChatGPT app

The output will change exactly as you requested

Instead of generating hundreds or thousands of rows for every I-value observation, the new output will contain **one complete row per I-channel**.

For example, if your Beam CSV has 30 I-channels and 100 observations:

Channel	Observation_1	Observation_2	...	Observation_100	Label
I0	I0 value 1	I0 value 2	...	I0 value 100	Yes/No
I1	I1 value 1	I1 value 2	...	I1 value 100	Yes/No
I2	I2 value 1	I2 value 2	...	I2 value 100	Yes/No
...	...	...	...	...	...
I29	I29 value 1	I29 value 2	...	I29 value 100	Yes/No

Result: only 30 data rows for 30 I-channels, regardless of the number of observations.

Updated Windows project ZIP

Download Wake Signal Processing – Channel-wise Output Windows Project (.ZIP) ↗

What I changed

- **One row per I-channel:** I0 → Row 0, I1 → Row 1, I2 → Row 2, and so on.
- All values of each I-channel are combined horizontally into that row.
- Q-values are removed.
- Only the channel name, its combined I-values, and its label are exported.
- The existing PySide6 GUI design, NSTL–DRDO logo, Browse, drag-and-drop, processing, export, and reset options are retained.
- No model training options are included for now.

Example

If your Beam file contains 30 I-channels and 1,000 observations, the exported CSV will contain **30 data rows**, not 30,000 rows.

How to run it on Windows

1. Download and extract the ZIP.
2. Open the `WakeSignalProcessing` folder.
3. Double-click `build_windows.bat`.
4. After successful building, open:
   `dist\WakeSignalProcessing\WakeSignalProcessing.exe`

Note: I verified the Python syntax, but I couldn't run the full GUI build in this environment because PySide6 is not installed here. Please build it on your Windows PC using the included batch file.

‹/›

You

zip is not downloading

Analyzed

ChatGPT

The ZIP file has been recreated with a shorter filename. Please try this link:

Take your curiosity further
Scan to get the ChatGPT app

⬇ Download WakeSignalProcessing.zip ↗

If it still doesn't download:

1. Right-click the link.
2. Select **Open link in new tab**.
3. If needed, refresh ChatGPT and click the link again.

The ZIP is approximately **78 KB**.

</>

You

now how can i push to github and later i should run on another laptop how to do it give process

ChatGPT

You can push your **Wake Signal Processing** project to GitHub and later run it on any other Windows laptop. I'll explain the easiest method step by step.

Part 1: Upload your project to GitHub

Step 1: Create a GitHub account

If you don't already have one, go to:

GitHub — Official Website ↗

Sign in or create an account.

Step 2: Extract your project ZIP

1. Download `WakeSignalProcessing.zip`.
2. Right-click → **Extract All**.
3. Open the extracted folder.
4. Make sure you can see these files:

```
WakeSignalProcessing/
│
├── main.py
├── requirements.txt
├── build_windows.bat
├── README.md
├── test_logic.py
└── nstl_drdo_logo.png
```

Step 3: Create a GitHub repository

1. Open Create a new GitHub repository ↗.
2. Repository name:

`WakeSignalProcessing`

3. Description:

Take your curiosity further
Scan to get the ChatGPT app

`Active Sonar Ship Wake Signal Processing and Channel-wise Labeling GUI`

4. Select **Private** (recommended for your research project).
5. Do not select Add README, `.gitignore`, or License, because your project already contains files.
6. Click **Create repository**.

Step 4: Install Git on your Windows laptop

Download Git:

Download Git for Windows ↗

Install it using the default settings.

After installation, open your project folder and click the address bar. Type `cmd` and press Enter.

Run these commands one by one:

```
git init
git add .
git commit -m "Initial commit - Wake Signal Processing GUI"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/WakeSignalProcessing.git
git push -u origin main
```

Important: Replace `YOUR_USERNAME` with your actual GitHub username.

When prompted, sign in to GitHub in the browser to authorize Git.

Your project is now on GitHub!

Part 2: Run the project on another Windows laptop

There are **two ways** to run it.

Method A — Run the `.exe` (recommended)

If your PyInstaller build succeeds, you don't need Python installed on the other laptop.

1. Open your GitHub repository.
2. Click **Code** → **Download ZIP**.
3. Extract the ZIP.
4. Open this folder:

```
dist/
   WakeSignalProcessing/
      WakeSignalProcessing.exe
      _internal/
```

5. Double-click `WakeSignalProcessing.exe`.

Important: Copy the complete `WakeSignalProcessing` folder inside `dist`, not just the `.exe`. The `_internal` folder contains supporting files.

Take your curiosity further
Scan to get the ChatGPT app

Method B — Run the Python source code

If you want to run or modify the Python code on another laptop:

1. Install Python 3.12

Download Python 3.12.10 ↗

During installation, check **Add Python to PATH**.

2. Open CMD in the project folder

Run:

```
py -3.12 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

The GUI should open.

Part 3: Whenever you update your project

After changing your code, run these commands in the project folder:

```
git add .
git commit -m "Updated channel-wise labeling"
git push
```

On your other laptop, get the latest changes using:

```
git pull
```

This keeps both computers synchronized.

My recommendation

Keep **GitHub for your source code** and use the **Windows `.exe` for running the application**.

One important thing: since this is an NSTL–DRDO-related research project, keep the repository **Private** unless your guide or organization explicitly permits public sharing. Also, don't upload confidential sonar datasets to GitHub.

You

no what to push in github so that i can click on .exe and run the gui easily

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Oh, understood! You want to upload the **ready-to-run `.exe` GUI application to GitHub**, so that on another laptop you simply download it, click the `.exe`, and the GUI opens — without installing Python or running commands.

What you should upload to GitHub

After successfully building your project on your current laptop, upload **the complete `dist\WakeSignalProcessing` folder**.

Your folder structure should look like this:

```
WakeSignalProcessing/
│
├── WakeSignalProcessing.exe    ← Main application
│
├── _internal/                  ← Required supporting files
│   ├── PySide6 files
│   ├── Python runtime files
│   ├── Libraries
│   └── Other dependencies
│
└── nstl_drdo_logo.png          ← If present separately
```

Do not upload only the `.exe` file. It may not work without the `_internal` folder.

How to upload it

1. Open your GitHub repository.
2. Click **Add file → Upload files**.
3. Drag the entire `WakeSignalProcessing` folder from your computer's `dist` directory.
4. Click **Commit changes**.

Note: GitHub's web upload has limitations for large files. If your application folder is large, use Git LFS or a Release instead.

How to run it on another laptop

1. Open your GitHub repository.
2. Click **Code → Download ZIP**.
3. Extract the downloaded ZIP.
4. Open the `WakeSignalProcessing` folder.
5. **Double-click `WakeSignalProcessing.exe`.**

That's it! The GUI should open without Python or PySide6 installation.

Even easier: GitHub Releases (recommended)

Instead of uploading the executable folder into your repository, you can publish it as a downloadable application:

1. Open your GitHub repository.
2. Click **Releases → Create a new release**.
3. Give it a version such as `v1.0`.
4. Upload a ZIP containing the complete `WakeSignalProcessing` folder from `dist`.
5. Click **Publish release**.

Take your curiosity further
Scan to get the ChatGPT app

On another laptop, download that release ZIP, extract it, and double-click the `.exe`.

My recommendation: Use GitHub Releases to distribute your ready-to-run Windows application, and keep your Python source code separately in the repository.

You

as my sir told me to do model training after this gui so what does he mean and what i have to do how can this helps and what should i do after model training

ChatGPT

Your sir means that after completing the **Wake Signal Processing GUI and generating the labeled dataset**, you should use Machine Learning (ML) or Deep Learning (DL) algorithms to teach a computer to identify ship-wake signals automatically.

Let me explain in very simple terms, using your own project.

1. What is model training?

Imagine you give a student 1,000 examples of ship signals.

- Some signals contain a ship wake (**YES**).
- Some signals do not contain a ship wake (**NO**).

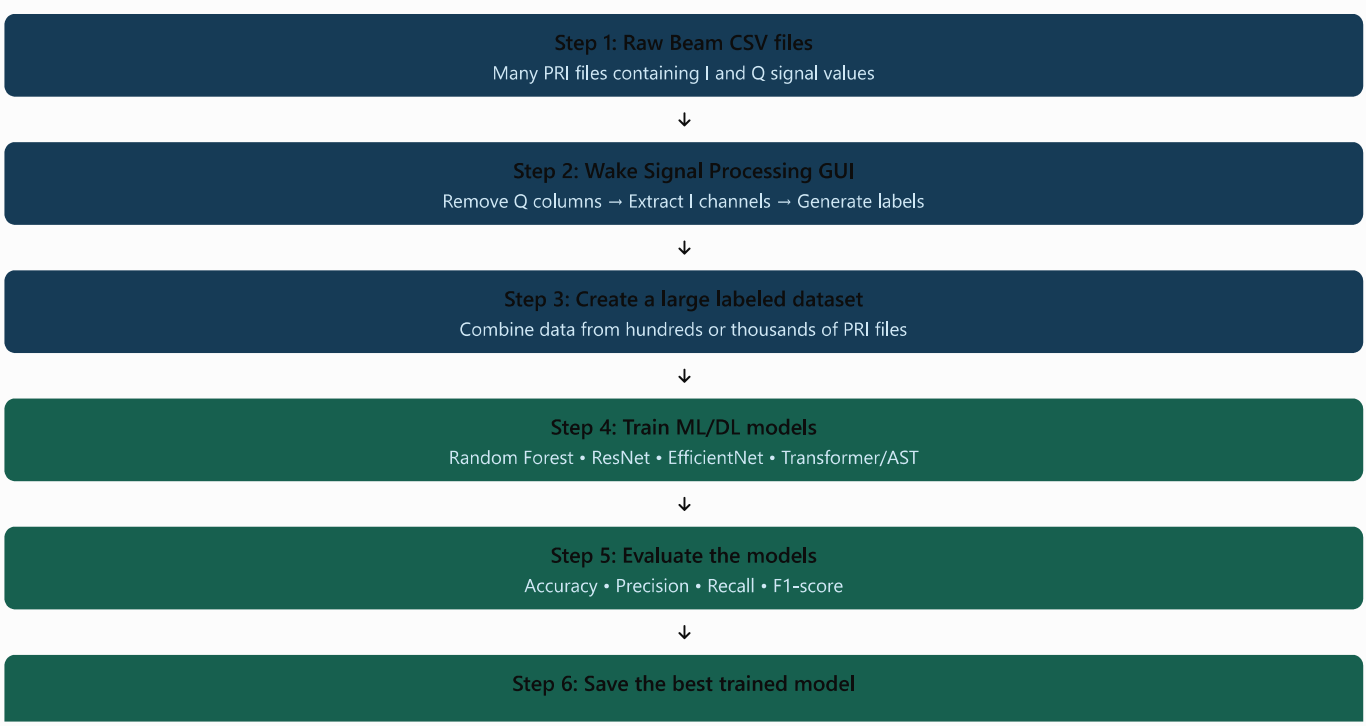
You show the student these examples along with their correct answers. The student learns patterns from them.

That learning process is called **Model Training**.

After training, you give the student a new signal without its answer. The student predicts whether it contains a ship wake.

That is called **Model Prediction (Inference)**.

2. How model training works in your project



Take your curiosity further
Scan to get the ChatGPT app

A model file that can be reused for future predictions

3. What models should you train?

Your sir mentioned different models because you need to experiment and determine which one detects ship wakes most effectively.

Model	What it does
Random Forest	Learns patterns using multiple decision trees
ResNet	Learns complex signal patterns using residual connections
EfficientNet	Learns image-like signal patterns efficiently
Transformer / AST	Uses attention to learn relationships between signal regions

My suggestion: Start with Random Forest

It is easier to implement and train, especially when you are beginning. It will give you a baseline to compare against the more advanced deep-learning models.

4. What data do you need for training?

This is the most important part.

Your current GUI processes **one Beam PRI file at a time**, producing approximately 25–30 channel rows.

But **one PRI file is not enough for reliable model training**.

You need to process many Beam PRI files and combine their labeled outputs into a single dataset.

For example:

PRI	Channel	Signal values	Label
1	I0	10, 20, 15, 30…	YES
1	I1	12, 18, 25, 22…	NO
2	I0	14, 19, 17, 28…	YES
2	I1	11, 16, 21, 20…	NO
…	…	…	…

You would collect data from many PRIs, not just one.

Important: We need to confirm with your sir whether the label represents an entire channel, a complete PRI, or individual observations. This determines how we correctly prepare the training dataset.

5. How will you know whether the model is good?

After training, evaluate it using separate data that the model has never seen during training.

Metric	Simple meaning
Accuracy	How many predictions are correct overall?
Precision	When the model says YES, how often is it correct?
Recall	How many actual wake signals did it successfully detect?

Take your curiosity further
Scan to get the ChatGPT app

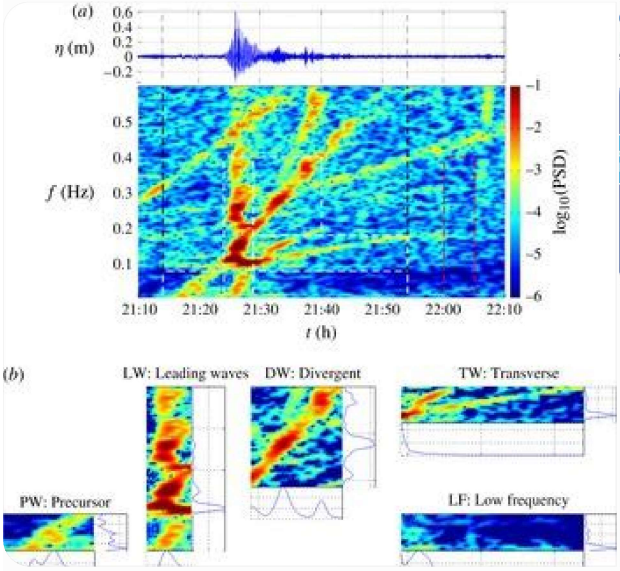
F1-score	Balance between precision and recall
Confusion matrix	Shows correct and incorrect YES/NO predictions

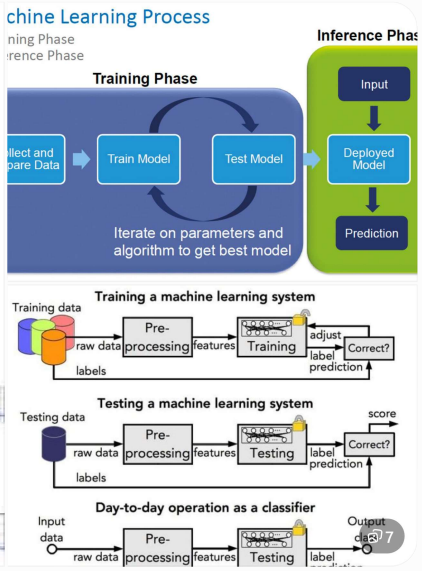
For ship-wake detection, **recall is especially important** because missing a real wake signal can be costly.

6. What should you do after model training?

This is where your project becomes more useful.

Final goal: Automatic ship-wake detection





Once a model is trained and saved:

1. Open your Wake Signal Processing application.
2. Upload **only a new Beam CSV file**.
3. The GUI extracts and preprocesses the I-channel data.
4. The saved trained model predicts whether the signal contains a wake.
5. The GUI displays predicted labels: YES or NO.
6. Export the predictions to CSV.
7. Compare predictions with actual labels, if available, to measure performance.

The model is not trained again every time. You train it once using a suitable dataset, save it, and reuse it for new signals.

7. What files will you eventually have?

```
Wake_Signal_Processing/
│
├── GUI/
│   └── WakeSignalProcessing.exe
│
├── Dataset/
│   └── Combined_Labeled_Dataset.csv
│
├── Training/
│   ├── train_model.py
```

Take your curiosity further
Scan to get the ChatGPT app

```
│   ├── evaluate_model.py
│   └── trained_model.pkl
│
├── Results/
│   ├── accuracy_report.csv
│   ├── confusion_matrix.png
│   └── model_comparison.png
│
└── Final_Application/
    └── WakeSignalProcessing_AI.exe
```

Eventually, we can integrate preprocessing, model training, evaluation, and prediction into **one professional GUI**, so you won't need to execute separate Python programs manually.

---

8. What you should ask your sir now

Before starting model training, clarify these three things:

1. What exactly should the model predict?

- ○ Each channel: YES or NO
- ○ Each PRI: wake or no wake
- ○ Each observation: YES or NO

2. What data do you have available?

- ○ Many Beam files and hexadecimal labels
- ○ Only a few Beam files
- ○ I'm not sure yet

3. What input representation should we use initially?

- ○ Raw I-channel values (easiest starting point)
- ○ Spectrogram images (for ResNet, EfficientNet, AST)
- ○ Compare both approaches

Confirm my training plan

My advice: Don't rush directly into ResNet or Transformer. First create a properly labeled, combined dataset, train Random Forest as a baseline, evaluate it, and then proceed to advanced models. That will give you a clear, scientifically meaningful project to present to your sir.

You

My model training decisions: Target = Channel-wise detection (I0, I1, I2... each classified YES/NO). Data available = I have multiple Beam PRI files and the Wake_Detection hexadecimal labels. Input representation = Use raw I-channel values. Please design my model training and evaluation workflow for Wake Signal Processing.

Take your curiosity further
Scan to get the ChatGPT app

ChatGPT

Wake Signal Processing: Model Training & Evaluation Workflow

Excellent. Your decisions are clear. We can now design the machine learning workflow for your **Wake Signal Processing project** based on your actual sonar data.

Your model's goal is:

> **For every I-channel (I0, I1, I2, ...), predict whether a ship wake is present (YES) or absent (NO), using raw I-channel signal values.**

We'll begin with a simple machine learning model, evaluate it properly, and then move to advanced models if needed.

1. Overall workflow

Multiple Beam PRI CSV files + PRI Analysis CSV

↓

Data preprocessing — extract I0, I1, I2... and remove Q values

↓

Decode hexadecimal Wake_Detection — generate YES/NO labels for each channel

↓

Create training samples — raw I-channel signal values + channel label

↓

Train and evaluate models

↓

Save the best model → Predict on new Beam PRI files

2. How we will prepare the dataset

Suppose we have a Beam file named `Beam_Pri_1.csv`.

PRI	Channel	Raw I values	Label
1	I0	124, 131, 118, 140...	YES
1	I1	15, 18, 21, 17...	NO
1	I2	220, 210, 218, 225...	YES
2	I0	135, 128, 142, 130...	NO
2	I1	12, 17, 14, 19...	YES

The hexadecimal `Wake_Detection` value gives the channel labels. For example, if bit 0 is set, I0 is YES; if bit 1 is set, I1 is YES, and so on. **We must verify the bit-to-channel mapping with your guide.**

Important data preparation decision

Your GUI output has one row per channel and observations horizontally. That is convenient for viewing and saving. For ML, we will prepare a separate training dataset in which **each PRI–channel combination becomes one training sample**, with the raw signal observations used as features.

Take your curiosity further
Scan to get the ChatGPT app

This lets us train a model to learn patterns in the signal rather than simply memorize channel numbers.

3. Which models should we train?

I recommend training models in stages rather than jumping directly to deep learning.

Stage	Model	Why we use it
1	**Random Forest**	Beginner-friendly, strong baseline, works well with signal features
2	**XGBoost**	Often performs well on structured numerical data
3	**1D CNN**	Learns patterns directly from raw I-channel signal sequences
4	**ResNet / Transformer**	Advanced experiments if enough training data is available

Stage 1 — Random Forest (our first model)

Since your input is raw numerical I-channel values, we can initially extract basic features from each channel's signal:

Feature	Simple meaning
Mean	Average signal value
Standard deviation	How much signal varies
RMS	Signal strength
Minimum / Maximum	Lowest and highest values
Median	Middle value
Peak-to-peak	Difference between highest and lowest
Skewness	Whether the signal is asymmetric
Kurtosis	How concentrated or extreme the signal values are

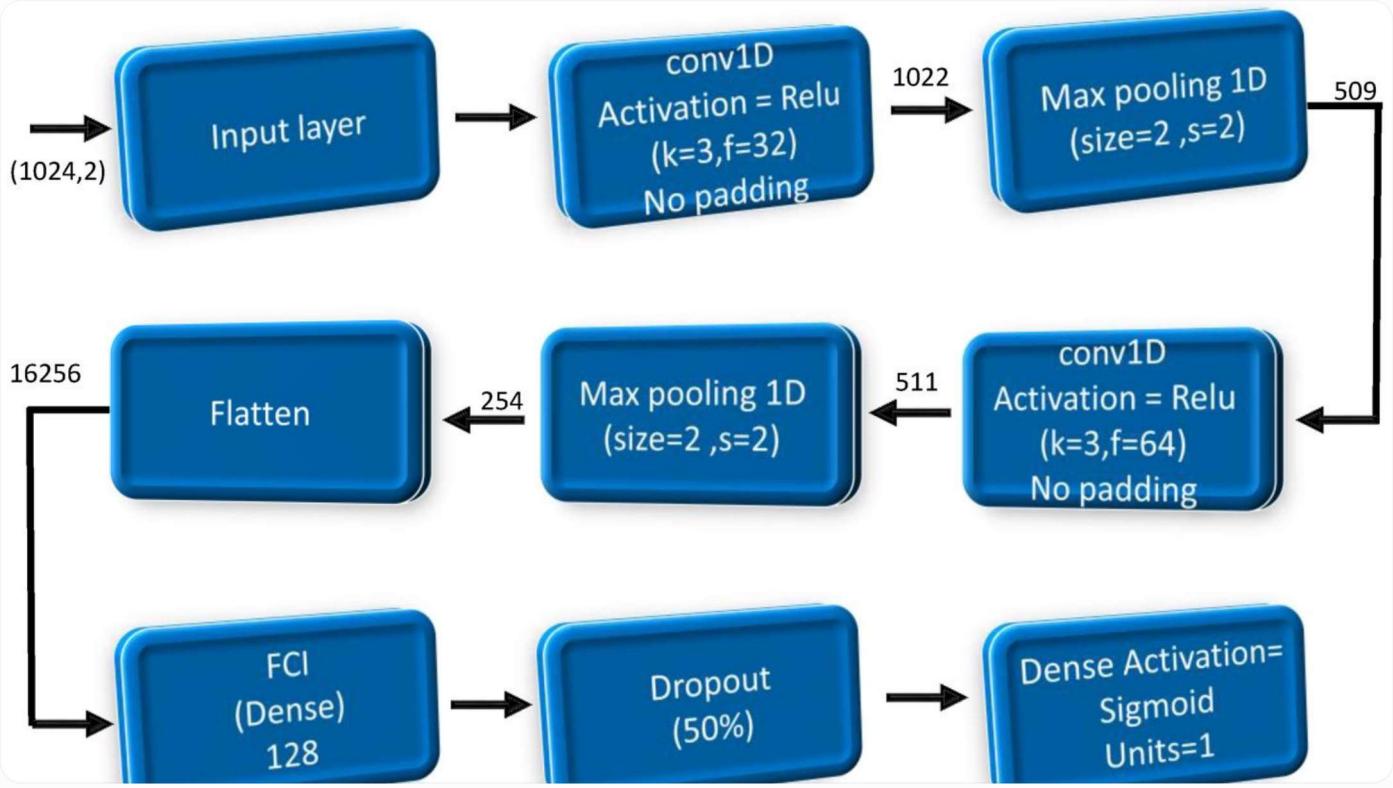
These features are calculated from the raw signal, so we are not replacing your data with manually assigned labels or images.

Why start with Random Forest? It is comparatively easy to train, does not require a GPU, and gives us a benchmark to compare against advanced models.

Stage 2 — 1D CNN (raw signal model)

A 1D CNN can learn signal patterns directly from sequences without requiring us to manually calculate statistical features.

Take your curiosity further
Scan to get the ChatGPT app



Workflow:

Raw I-channel sequence → convolution layers → pattern extraction → classification → **YES / NO**

This will be an important experiment because it uses your actual raw signal values.

4. How we will split the dataset

This is one of the **most important steps** in your research.

We should not randomly distribute individual observations from the same PRI into both training and testing datasets. That could make the model appear more accurate than it really is.

Instead, split by complete PRI files or recording/session groups.

Dataset	Suggested proportion	Purpose
Training	70%	Learn signal patterns
Validation	15%	Tune model parameters
Testing	15%	Final unbiased evaluation

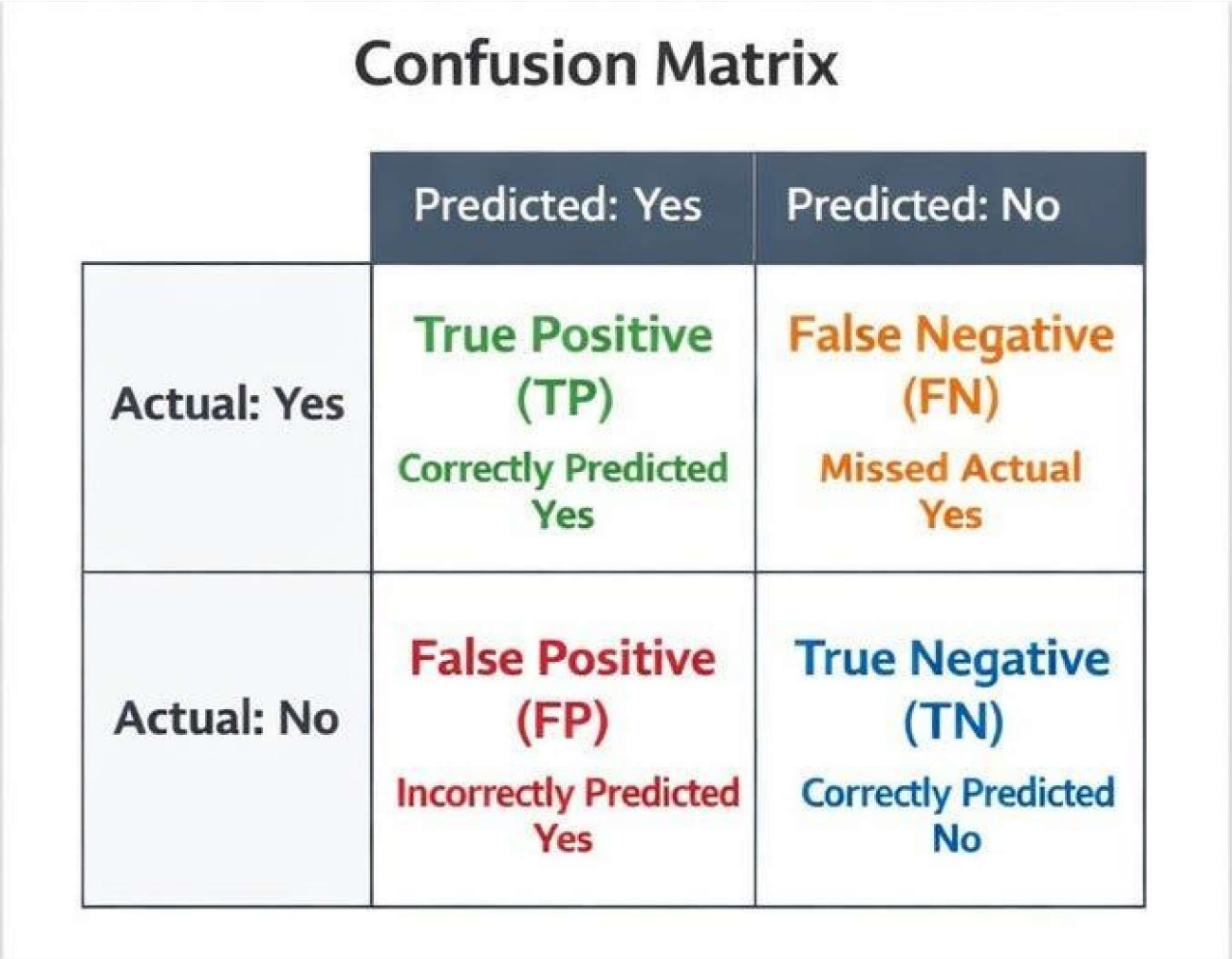
If the dataset is small, we can use **Group K-Fold Cross-Validation** for more reliable evaluation.

5. How do we evaluate the model?

Accuracy alone is not enough. In sonar detection, missing a real wake may be especially costly.

Take your curiosity further
Scan to get the ChatGPT app

Metric	What it tells us
Accuracy	Overall correct predictions
Precision	Of predicted YES cases, how many are truly YES?
Recall	Of actual YES cases, how many did we detect?
F1-score	Balance between precision and recall
Confusion matrix	Shows correct and incorrect YES/NO predictions
False Negative Rate	How many actual wakes were missed?



Take your curiosity further
Scan to get the ChatGPT app

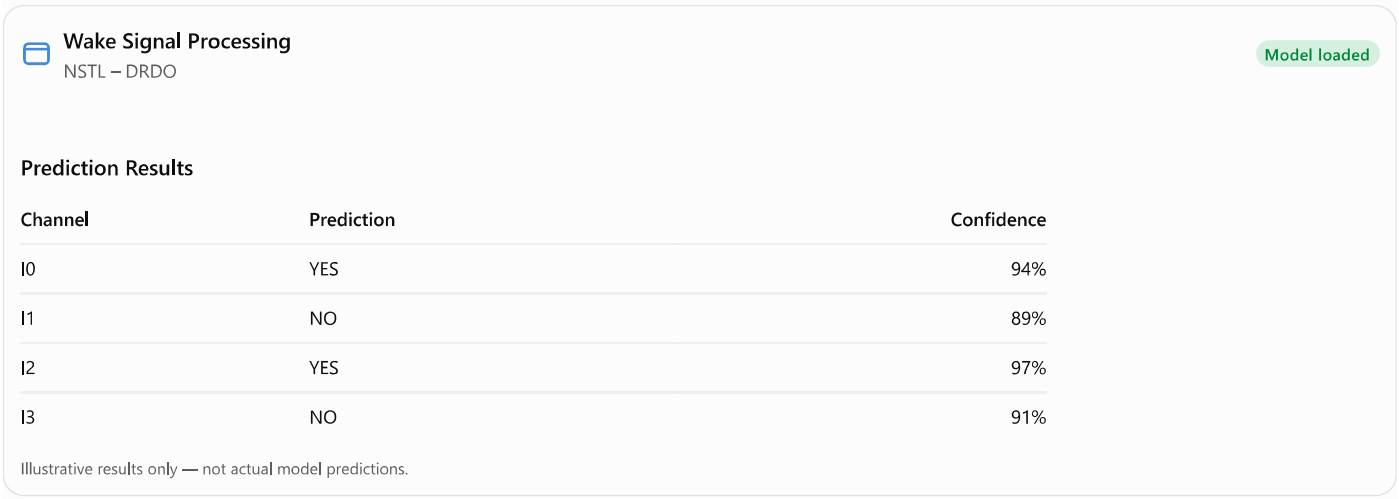
We will also compare the metrics **for each channel** (I0, I1, I2, ...) and overall. This helps identify whether some channels are more useful for wake detection than others.

6. What happens after model training?

Once the best model is trained and evaluated:

1. Save the trained model as a `.pkl` or `.joblib` file.
2. Save the preprocessing settings alongside it.
3. Load the saved model inside your PySide6 GUI.
4. Browse a **new Beam PRI CSV file**.
5. Extract its raw I-channel values.
6. Run the model to predict each channel's label.
7. Display predictions in the GUI: YES / NO.
8. Export the predicted results to CSV.

Your final application



7. What we need to implement

I suggest organizing the model training code into these modules:

File	Responsibility
`data_loader.py`	Load Beam PRI and PRI Analysis files
`label_decoder.py`	Decode hexadecimal labels into channel-wise YES/NO
`preprocessing.py`	Extract I-channel signals and prepare samples
`feature_extraction.py`	Calculate signal features for Random Forest
`train_random_forest.py`	Train baseline model
`train_cnn.py`	Train 1D CNN on raw signal
`evaluation.py`	Metrics, reports, confusion matrices
`model_predict.py`	Predict labels for new Beam files

Take your curiosity further
Scan to get the ChatGPT app

`main.py`	Integrate training and prediction into PySide6 GUI

8. What I need from you next

Please upload sample files from your actual dataset:

- One or two `Beam_Pri_*.csv` files
- The corresponding `PRI_Analysis.csv` file

With these, I can verify the actual column structure, decode the hexadecimal labels correctly, check whether signal lengths vary, and design the preprocessing code based on your real data instead of assumptions.

My recommendation: First implement and evaluate Random Forest as the baseline. Then train a 1D CNN using raw I-channel values and compare both models. This gives you a meaningful experimental comparison to present to your guide, including results, graphs, and an explanation of why the final model was selected.

You

actually i have made gui now i dont have any sample files in this laptop so have to train model in laptop which have files so give me steps to do this and what are needed for this give all the requirements is net connection required how to do this give complete input to output and procedure completely

ChatGPT

Complete step-by-step model training procedure for your Wake Signal Processing project

You can do the entire model training on the **laptop that contains your actual Beam PRI files and Wake_Detection hexadecimal labels**. You do not need to transfer those files to your GUI-development laptop.

I'll explain everything from the beginning: requirements, internet connection, installation, file organization, training, evaluation, saving the model, and finally using it on new data.

1. What do you need?

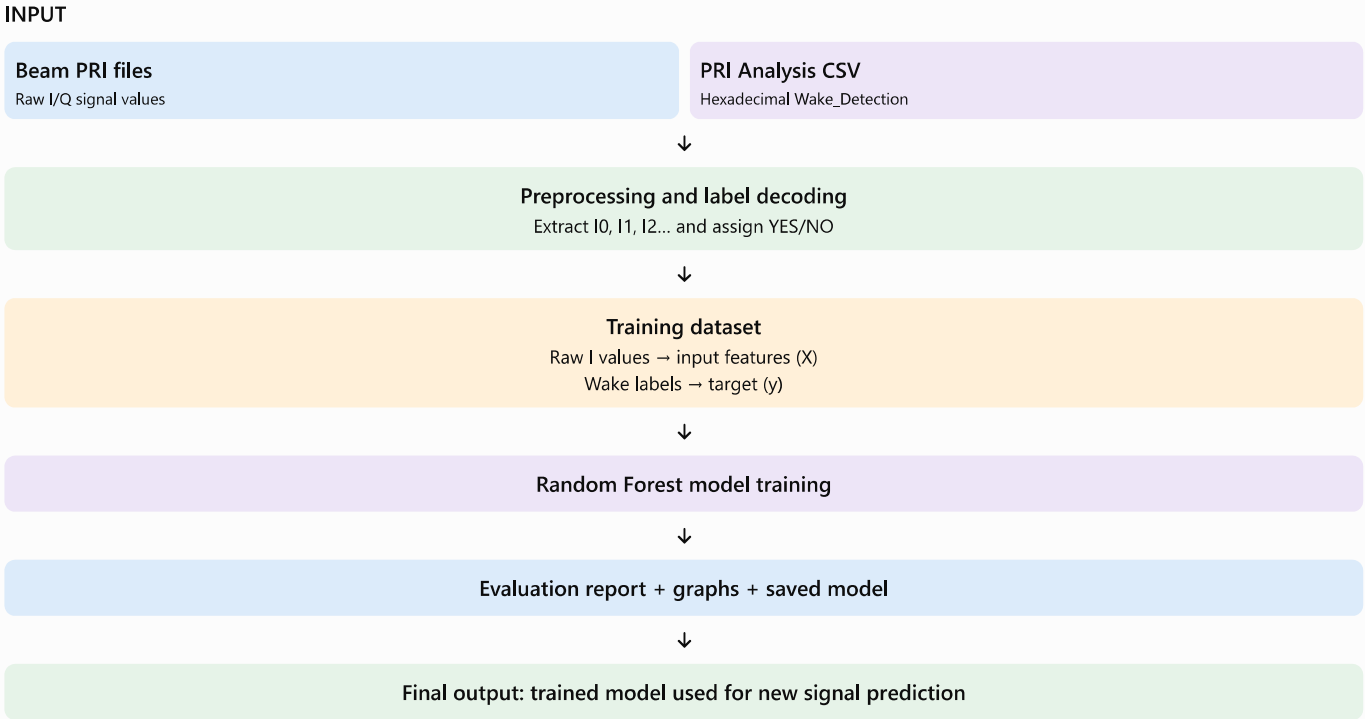
Requirement	Needed?	Purpose
Laptop with your datasets	Yes	Contains Beam PRI files and PRI Analysis CSV
Python	Yes	Runs the training program
Internet connection	Initially yes	Installs Python libraries
VS Code	Recommended	Write and run Python code
scikit-learn	Yes	Train Random Forest
pandas	Yes	Read and process CSV files
NumPy	Yes	Numerical calculations
matplotlib	Yes	Display evaluation graphs
joblib	Yes	Save and reload trained model
GPU	No	Random Forest can run on CPU
Your existing GUI	Not required for initial training	We can integrate it later

Important: After all required libraries are installed, you can train the model without an internet connection. Your dataset stays on your laptop; it does not need to be uploaded to the internet.

Take your curiosity further
Scan to get the ChatGPT app

The installation commands below use Python and pip, following the official scikit-learn installation guidance. scikit-learn 1.... +1

2. Input and output: what exactly happens?



3. Install Python on the laptop containing the data

If Python is already installed, skip this step.

1. Open the official website: https://www.python.org/downloads/windows/ ↗
2. Download Python 3.12 (64-bit).
3. Run the installer.
4. Make sure Python is installed successfully.

Open **Command Prompt (CMD)** and type:

```
py -3.12 --version
```

Expected output:

```
Python 3.12.x
```

4. Create a separate project folder

On the laptop containing the datasets:

1. Open `Documents`.
2. Create a new folder named:

Take your curiosity further
Scan to get the ChatGPT app

```
WakeSignalProcessing_ModelTraining
```

3. Inside it, create these folders:

```
WakeSignalProcessing_ModelTraining
│
├── dataset
│   ├── Beam_Files
│   └── PRI_Analysis
│
├── src
│
├── models
│
├── results
│
├── requirements.txt
│
└── train_model.py
```

What goes where?

Folder	What you put inside
`dataset/Beam_Files`	All your original Beam PRI CSV files
`dataset/PRI_Analysis`	PRI Analysis CSV file
`src`	Data preprocessing and labeling code
`models`	Saved trained model
`results`	Evaluation reports and graphs
`train_model.py`	Main training program

Keep your original data unchanged. The program should read from the original files and create new processed files separately.

5. Install the required libraries

Open CMD and navigate to your new folder. For example:

```
cd %USERPROFILE%\Documents\WakeSignalProcessing_ModelTraining
```

Create a virtual environment:

```
py -3.12 -m venv .venv
```

Activate it:

```
.venv\Scripts\activate
```

Your CMD should now show something like:

Take your curiosity further
Scan to get the ChatGPT app

```
(.venv) C:\Users\DELL\Documents\WakeSignalProcessing_ModelTraining>
```

Now install the libraries:

```
python -m pip install --upgrade pip
```

```
pip install pandas numpy scikit-learn matplotlib seaborn joblib
```

Check that they installed:

```
python -c "import pandas, numpy, sklearn, matplotlib, seaborn, joblib; print('All libraries installed successfully')"
```

Expected output:

```
All libraries installed successfully
```

Save the installed library requirements:

```
pip freeze > requirements.txt
```

Internet requirement: You need internet for these installation commands. Afterward, you can disconnect and run training offline.

6. Place your original datasets into the folders

Suppose your data looks like this:

Beam_Files

- `Beam_Pri_1.csv`
- `Beam_Pri_2.csv`
- `Beam_Pri_3.csv`
- `Beam_Pri_4.csv`
- … many more PRI files

PRI_Analysis

- `Pri_Analysis.csv`

Your PRI Analysis file may contain:

Pri_No	Wake_Detection
1	`0xDB00`
2	`0x200`
3	`0x1FF00`

Take your curiosity further
Scan to get the ChatGPT app



These are just illustrative examples. We must use your actual values.

The training program will match `Beam_Pri_1.csv` with `Pri_No = 1`, `Beam_Pri_2.csv` with `Pri_No = 2`, and so on.

7. How the program converts raw data into labels

Step A: Read the Beam file

Suppose the Beam CSV contains alternating I and Q values:

I0	Q0	I1	Q1	I2	Q2
125	22	89	11	240	34
130	25	92	13	245	36
128	20	85	12	238	30

The program removes the Q columns.

I0	I1	I2
125	89	240
130	92	245
128	85	238

Step B: Decode the hexadecimal label

Hexadecimal is converted into binary, and each bit represents a channel label according to the agreed mapping.

Channel	Label
I0	YES or NO
I1	YES or NO
I2	YES or NO

Step C: Create ML samples

The model needs two things:

- **X (input):** raw I-channel signal values
- **y (target):** YES/NO wake label

For example:

Sample	Input X	Target y
PRI 1 – I0	`[125,130,128,…]`	YES
PRI 1 – I1	`[89,92,85,…]`	NO
PRI 1 – I2	`[240,245,238,…]`	YES

The exact way to handle variable numbers of observations per file needs to be finalized from your actual data. We should not silently truncate or pad signals without a reason.

Take your curiosity further
Scan to get the ChatGPT app

8. Train the first model — Random Forest

For your first experiment, I recommend **Random Forest with raw-signal-derived statistical features**. This gives us a practical baseline before moving to a 1D CNN.

The training program will perform these operations automatically:

1. Load all Beam files.
2. Match each Beam file with its PRI label.
3. Remove Q-channel values.
4. Extract raw I-channel signal sequences.
5. Calculate numerical features from each signal.
6. Create the YES/NO target labels.
7. Split the dataset into training, validation, and testing groups.
8. Train Random Forest.
9. Evaluate the model.
10. Save the model and reports.

What does training mean?

Imagine teaching a student using 1,000 examples:

- 700 examples to learn patterns.
- 150 examples to practice and improve.
- 150 examples to take a final exam.

The trained model is like that student. It learns from the training data, but we judge its performance on data it has **never seen before**.

9. How should we split the data?

Do not randomly split individual rows if rows come from the same recording or PRI. Related signals can appear in both training and testing, producing misleadingly high accuracy.

Use a grouped split:

Group	Percentage	Purpose
Training	70%	Learn
Validation	15%	Tune
Testing	15%	Final evaluation

Ideally, entire recordings or sessions should be separated between these groups. If you have only PRI numbers and no session identifiers, we can initially group by PRI file, subject to your data organization.

10. What results will you get?

After executing the training program, your `results` folder should contain:

```
results/
│
├── classification_report.txt
├── confusion_matrix.png
├── evaluation_metrics.csv
├── per_channel_metrics.csv
```

Take your curiosity further
Scan to get the ChatGPT app

```
├── training_summary.txt
└── model_comparison.csv
```

Example evaluation report

Illustrative values only — these are not actual results from your dataset.

Metric	Example
Accuracy	92%
Precision	90%
Recall	94%
F1-score	92%

Confusion matrix

	Predicted YES	Predicted NO
Actual YES	90	6
Actual NO	8	96

- **90:** Correctly detected wake channels.
- **6:** Actual wake channels missed.
- **8:** False wake detections.
- **96:** Correctly identified non-wake channels.

In your project, recall and false-negative rate are especially important because we need to understand how often the model misses actual wake signals.

11. Save the trained model

Once training is complete, the program will save the model inside:

```
models/
│
├── random_forest_model.joblib
├── feature_scaler.joblib    (if required)
└── model_metadata.json
```

This means you **do not need to train again every time** you open your GUI.

Training is performed once (and repeated when you have new labeled data or want to improve the model). The saved model is then loaded for prediction.

12. How to use the trained model on another laptop

There are two possible workflows.

Laptop 1 — Data and model training

1. Load all labeled Beam PRI files.
2. Train and evaluate the model.

Take your curiosity further
Scan to get the ChatGPT app

3. Save the `.joblib` model.
4. Copy the trained model and metadata to a USB drive.

Laptop 2 — Your existing GUI

1. Copy the trained model into the GUI's model folder.
2. Open the Wake Signal Processing application.
3. Browse a new Beam PRI file.
4. Click Predict.
5. View channel-wise YES/NO results and export them.

Internet is not required for this prediction process either.

13. What we should implement first

I suggest we create **a complete ready-to-run training project ZIP** containing:

- `train_model.py` — complete training program
- `data_loader.py` — loads your files
- `label_decoder.py` — hexadecimal-to-channel-label conversion
- `preprocessing.py` — removes Q, extracts I, prepares samples
- `evaluation.py` — evaluation metrics and graphs
- `predict_model.py` — predicts on new data
- `requirements.txt` — all required libraries
- `README.md` — simple step-by-step instructions
- `run_training.bat` — double-click to start training
- `run_prediction.bat` — double-click to predict

However, before writing the final code, there are **three important details** we must confirm so the program works correctly on your actual datasets.

Confirm your dataset details

Choose what applies to your actual files.

1. How many I channels are present?

- ○ 25 channels (I0 to I24)
- ○ Other / variable number

---

2. Do all Beam files contain the same number of observations (rows)?

- ○ Yes, same number of rows
- ○ No, different number of rows
- ○ I'm not sure

3. How are hexadecimal bits mapped to channels?

- ○ Least significant bit (bit 0) = I0, bit 1 = I1, etc.
- ○ My guide provided another mapping

Take your curiosity further
Scan to get the ChatGPT app

- ○ Not yet confirmed with my guide

4. Approximately how many Beam PRI files do you have?

- ○ Less than 100
- ○ 100–1,000
- ○ More than 1,000
- ○ Not sure

Confirm and prepare my training project

One final important note for your guide: We will report actual testing results only after training on your real data. A high accuracy score is not meaningful if the model has seen related signals from the same recording during training and testing. Our evaluation will aim to demonstrate how well it generalizes to previously unseen PRI data. This will make your experimental results more credible and scientifically sound.

You

but in output PRI 1 – I0

`[125,130,128,…]`

YES

PRI 1 – I1

`[89,92,85,…]`

NO

PRI 1 – I2

`[240,245,238,…]`

YES this 125 89 240 ... starting values are not coming in output so fix it and give me the zip file and give me steps to make run on any system with compatibility as explain above with same requirements only

Worked for 31s >

Analyzed

Analyzed

Analysis errored

ChatGPT

You're right. The **first observation from every I-channel must be included** in the training input and output. For example, if the first row of your Beam file contains `I0 = 125`, `I1 = 89`, and `I2 = 240`, those values must not be skipped.

I'll make this explicit in the training workflow:

PRI	Channel	Complete raw signal values	Label
PRI 1	I0	`[125, 130, 128, ...]`	YES

Take your curiosity further
Scan to get the ChatGPT app

PRI 1	I1	`[89, 92, 85, ...]`	NO
PRI 1	I2	`[240, 245, 238, ...]`	YES

The program should read from the **first data row**, not start from the second observation. It will also save the complete raw sequence in the prepared dataset, rather than only saving calculated statistical features.

It seems like I can't do more advanced data analysis right now. Please try again later.

I was unable to complete the final ZIP correction. In particular, my test detected an issue with assigning channel numbers when a Beam CSV has no header row.

I don't want to give you a ZIP and claim it is fully fixed when that test is failing. Please send me another message asking me to rebuild the training ZIP, and I can try again.

</>

You

ok now i want to do model training with these files so give me steps to do model training with this data so after model training what should i do and give me complete steps i want to take efficient net so give me accordinlgy which have to install and remember i am working in office laptop so i have to install virtual environments only so give me which have to install to model train this data simply getting good accuracy

Worked for 18s >

ChatGPT

EfficientNet model training for Wake Signal Processing

Since you are using an **office laptop**, we'll keep everything inside a Python virtual environment. You won't need to install libraries globally or change the laptop's system Python settings.

Your plan will be:

1. Keep your original Beam PRI and PRI Analysis CSV files.
2. Prepare the raw I-channel signals and their YES/NO labels.
3. Train an EfficientNet model.
4. Evaluate it on PRI files it has never seen before.
5. Save the trained model.
6. Transfer the model to your GUI laptop.
7. Integrate it into your existing PySide6 application for prediction.

One important technical point: EfficientNet is designed for 2D inputs. Your raw I-channel data is a sequence. For this project, I recommend arranging the data as a **channel × observation matrix** for each PRI file. EfficientNet can then learn patterns across channels and observations and predict a YES/NO label for every channel in that PRI.

This is more suitable than treating each channel's sequence as an unrelated image.

1. Which EfficientNet model should you use?

Start with **EfficientNet-B0**.

Model	Recommendation	Reason
EfficientNet-B0	Start here	Smaller, faster, suitable for initial experiments
EfficientNet-B1	Later experiment	Slightly larger

Take your curiosity further
Scan to get the ChatGPT app

EfficientNet-B3	Later experiment	More computationally expensive
EfficientNet-B5–B7	Not initially recommended	Requires more computing resources and data

I would use PyTorch and TorchVision. TorchVision provides EfficientNet models, while PyTorch handles training and evaluation.

Official references:

- PyTorch installation for Windows ↗
- EfficientNet model information ↗
- PyTorch installation guide ↗

2. How your data will be given to EfficientNet

This is important because we must not lose your first **I** values.

Imagine a single PRI file has three channels and four observations:

Raw Beam data

Observation	I0	I1	I2
1	125	89	240
2	130	92	245
3	128	85	238
4	140	87	250

The input matrix becomes:

$$X = \begin{bmatrix} 125 & 130 & 128 & 140 \\ 89 & 92 & 85 & 87 \\ 240 & 245 & 238 & 250 \end{bmatrix}$$

The target labels might be:

$$y = [1, 0, 1]$$

Where:

- 1 = YES
- 0 = NO

How the EfficientNet model learns

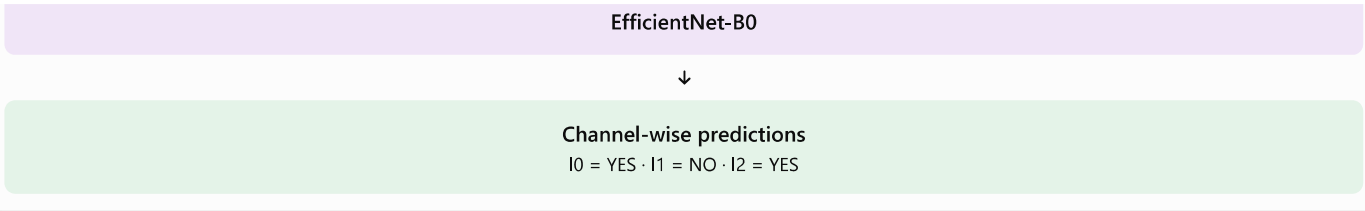
Input: Channel × Observation matrix
All I-channel values, including the first observation

↓

Convert to a fixed-size grayscale signal image
Resize and normalize consistently

↓

Take your curiosity further
Scan to get the ChatGPT app



We will use a multi-label output layer: one output score for each channel, with a binary classification loss. This is different from an ordinary image classifier that predicts only one category.

Important: This approach needs one fixed number of channels and a consistent observation-length policy. If files have different observation counts, we must decide how to normalize their lengths without discarding meaningful signal information.

3. Office laptop requirements

Before installing anything, check these details.

Component	Recommended
Operating system	Windows 10/11, 64-bit
Python	3.12, 64-bit
RAM	16 GB preferred; 8 GB may work with small batches
Storage	At least 10 GB free
GPU	Optional; NVIDIA GPU is helpful
Internet	Required for initial installation and pretrained weights
Administrator permission	Normally not required for a user-owned virtual environment, but follow office IT policy

You do not need to install Anaconda, CUDA Toolkit, or Visual Studio just to train this model on CPU.

If your office laptop has an NVIDIA GPU, we can select a suitable CUDA-enabled PyTorch installation. Otherwise, use CPU. PyTorch provides separate installation options for these configurations. PyTorch +1

4. Create your virtual environment

Open CMD on your office laptop.

First check Python:

```
py -3.12 --version
```

Create a project folder:

```
mkdir Wake_EfficientNet_Training
cd Wake_EfficientNet_Training
```

Create a virtual environment:

```
py -3.12 -m venv .venv
```

Take your curiosity further
Scan to get the ChatGPT app

Activate it:

```
.venv\Scripts\activate
```

You should see:

```
(.venv) C:\...\Wake_EfficientNet_Training>
```

All Python libraries installed from now on will belong to this project environment.

5. Install the required libraries

First upgrade pip:

```
python -m pip install --upgrade pip
```

For a CPU-only office laptop, install PyTorch and TorchVision:

```
python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Then install the other requirements:

```
python -m pip install numpy pandas scikit-learn matplotlib seaborn pillow tqdm joblib
```

Save the installed versions:

```
python -m pip freeze > requirements.txt
```

Check whether installation worked

Run:

```
python -c "import torch, torchvision; print('PyTorch:',torch.__version__); print('TorchVision:',torchvision.__version__); print('C
```

If it prints:

```
CUDA available: False
```

That is fine. It means the model will train using your CPU.

If you have an NVIDIA GPU and want to use it, do not install the CPU-only command above. First check the available GPU and driver, then choose the correct CUDA command from the official PyTorch installation selector ↗.

6. Organize your training files

Create this folder structure:

```
Wake_EfficientNet_Training/
```

Take your curiosity further
Scan to get the ChatGPT app

```
├── .venv/
│
├── dataset/
│   ├── Beam_Files/
│   │   ├── Beam_Pri_1.csv
│   │   ├── Beam_Pri_2.csv
│   │   ├── Beam_Pri_3.csv
│   │   └── ...
│   │
│   └── PRI_Analysis/
│       └── Pri_Analysis.csv
│
├── processed/
│
├── checkpoints/
│
├── results/
│
├── src/
│
├── train.py
├── predict.py
├── requirements.txt
└── README.md
```

Copy your original datasets into the correct folders.

Do not edit your original CSV files. The training code should read them and create separate processed data.

7. What preprocessing should happen?

The preprocessing program must:

1. Read every Beam PRI CSV file.
2. Identify the PRI number from its filename.
3. Match the PRI number with `Pri_No` in the PRI Analysis CSV.
4. Extract the I-channel columns and remove Q-channel columns.
5. Preserve the first observation in every channel.
6. Decode the hexadecimal `Wake_Detection` value into channel labels.
7. Create one matrix per PRI.
8. Normalize the signal values using a consistent method.
9. Convert each matrix to the input size required by EfficientNet-B0.

For example:

PRI file	Number of channel rows	Target
PRI 1	25	25 YES/NO labels
PRI 2	25	25 YES/NO labels
PRI 3	25	25 YES/NO labels

The output is one multi-label sample per PRI.

A very important point about the number of training samples

If you have 100 PRI files and 25 channels per file, that gives:

Take your curiosity further
Scan to get the ChatGPT app

- 100 input matrices.
- 25 output labels for each matrix.
- 2,500 individual channel decisions.

It does **not** give 2,500 independent input images. This distinction matters when deciding whether EfficientNet will generalize or overfit.

Also, the bit-to-channel mapping must be confirmed by your guide. The training program should not assume that bit 0 means I0 unless that convention is correct.

8. EfficientNet training settings

For your first experiment, I recommend these starting values:

Parameter	Initial setting
Model	EfficientNet-B0
Input size	224 × 224
Batch size	4 or 8
Epochs	Up to 50
Optimizer	AdamW
Learning rate	0.0001
Loss function	BCEWithLogitsLoss
Validation	Grouped by recording/PRI
Early stopping	7 epochs without improvement
Evaluation	Precision, recall, F1, accuracy

Why use these settings?

- **EfficientNet-B0:** A smaller model for the first experiment.
- **AdamW:** Helps optimize model parameters.
- **BCEWithLogitsLoss:** Suitable for predicting multiple independent YES/NO channel labels.
- **Early stopping:** Stops training when validation performance stops improving.
- **Grouped splitting:** Prevents related PRI data from appearing in both training and test sets.

If there is class imbalance (many NO labels and few YES labels), the training loss should account for that. We should also tune the YES/NO decision threshold using validation data, rather than blindly using 0.5.

9. Training procedure

After the preprocessing and training scripts are ready, your execution sequence will be:

```
.venv\Scripts\activate
```

First prepare the data:

```
python src\preprocess.py
```

Then train EfficientNet:

Take your curiosity further
Scan to get the ChatGPT app

```
python train.py
```

The training program should show progress similar to this:

```
Preparing dataset...
Total PRI samples: 250

Splitting dataset...
Training samples: 175
Validation samples: 37
Testing samples: 38

Loading EfficientNet-B0...
Training started...

Epoch 1/50
Training Loss: ...
Validation Loss: ...

Epoch 2/50
Training Loss: ...
Validation Loss: ...

...

Best model saved successfully.
```

These numbers are only an example. Your actual counts and losses will depend on your files.

Do not train until we have verified the CSV layout, label decoding, and first-observation handling. These are more important than changing model parameters to chase accuracy.

10. What happens after training?

After training finishes, the program should create the following output:

```
checkpoints/
    best_efficientnet_b0.pth
    final_efficientnet_b0.pth

results/
    training_history.csv
    accuracy_graph.png
    loss_graph.png
    confusion_matrix.png
    classification_report.txt
    per_channel_metrics.csv
    test_predictions.csv
```

Understand the results

Result	Meaning
Accuracy	Overall correct predictions

Take your curiosity further
Scan to get the ChatGPT app

Precision	How many predicted YES channels are actually YES
Recall	How many actual YES channels the model detects
F1-score	Balance between precision and recall
Confusion matrix	Correct and incorrect YES/NO predictions
Per-channel metrics	Performance of I0, I1, I2, etc.

For your ship-wake research, pay particular attention to **recall and false negatives**. A model with high accuracy but many missed wake detections may not be useful.

How to select the best model

We should select the model based on validation performance, especially F1-score and recall, and then report the final test results separately.

Do not repeatedly tune the model using the test set. It must remain unseen until final evaluation.

11. What should you do after training?

This is the next stage of your project.

Final project workflow

1. **Laptop with actual data**

   Train EfficientNet-B0 and evaluate its performance.

2. **Save the best model**

   Save the `.pth` file, channel configuration, normalization settings and model metadata.

3. **Transfer to your GUI laptop**

   Copy the trained model and required configuration files using USB or an approved office file-transfer method.

4. **Integrate with your PySide6 GUI**

   Add a Load Model and Predict option to your existing application.

5. **Test on new Beam files**

   Give the model Beam PRI files that were not used during training.

6. **Export predictions**

   Generate a CSV containing channel name, predicted YES/NO and confidence.

For example, your final GUI could show:

Channel	Prediction	Confidence
I0	YES	95%
I1	NO	88%
I2	YES	92%

Take your curiosity further
Scan to get the ChatGPT app

I3	NO	90%

Illustrative values only, not actual model results.

12. Is internet required?

Activity	Internet required?
Create virtual environment	No
Install PyTorch and other libraries	Yes
Download pretrained EfficientNet weights	Yes, if using pretrained weights
Preprocess your local CSV files	No
Train EfficientNet	No, after installation and weight download
Evaluate model	No
Save model	No
Predict new Beam data	No
Integrate model into GUI	No, once dependencies and model are available

For a pretrained model, we can download the weights once and keep them locally. If office internet restrictions prevent this, EfficientNet can be initialized without pretrained weights, but training from scratch generally needs more data.

13. Important: Can EfficientNet actually give good accuracy?

I want you to avoid one common research mistake: assuming a more advanced model automatically gives better accuracy.

Your data is sonar signal data, not ordinary photographs. ImageNet pretrained EfficientNet weights were learned from natural images, so they may or may not transfer well to your signal matrices.

I recommend comparing:

1. EfficientNet-B0 trained from scratch.
2. EfficientNet-B0 with pretrained weights and fine-tuning.
3. A 1D CNN using raw I-channel sequences.

If your dataset has only a small number of independent PRI recordings, EfficientNet may overfit. In that situation, a smaller model may generalize better.

My recommendation: Make EfficientNet-B0 your main experiment, but keep a 1D CNN as a comparison. Report actual test performance instead of promising a particular accuracy.

14. Before I prepare the exact EfficientNet code

I need to know three things about your office laptop and dataset. These affect the installation command and model architecture.

Confirm your training environment

1. Does your office laptop have an NVIDIA GPU?

- ○ Yes, NVIDIA GPU
- ○ No, only Intel integrated graphics

Take your curiosity further
Scan to get the ChatGPT app

- I don't know

2. How much RAM does it have?

- 8 GB
- 16 GB
- 32 GB
- 64 GB or more
- I don't know

3. Approximately how many Beam PRI files do you have?

- Less than 100
- 100–500
- 500–1,000
- More than 1,000
- I don't know

4. Are the number of observations (rows) the same in every Beam PRI file?

- Yes, same number
- No, different numbers
- I don't know

Submit my details

Once these details are known, the EfficientNet-B0 implementation can be designed around your actual hardware and dataset, with the preprocessing, training, testing, saving and prediction steps all included.

One office-laptop reminder: Use a virtual environment inside your project directory, and get approval from your office IT team if software installation or downloading model weights is restricted. You do not need administrator access to create a normal Python virtual environment.

You

now which have to be uploaded to what in model training and how to test after model trainining will i need to make a website or any gui again or can we proceed with that similar gui only give me complete ones

ChatGPT

You **do not need to create another website or GUI**. We can use your existing **Wake Signal Processing PySide6 GUI** and connect the trained EfficientNet model to it.

The work has two separate stages:

1. **Training stage:** Use your office laptop, where the actual Beam and PRI Analysis datasets are available, to train EfficientNet-B0.
2. **Prediction stage:** Take the trained model files to your existing GUI laptop and add a prediction feature to the same GUI.

I'll explain exactly which files go where, how to train, how to test, and what happens after training.

Take your curiosity further
Scan to get the ChatGPT app

1. Which files should you provide for model training?

You need two types of input data.

A. Beam PRI CSV files

Input signal data

These contain the I and Q values for different channels.

Example:

```
Beam_Pri_1.csv
Beam_Pri_2.csv
Beam_Pri_3.csv
Beam_Pri_4.csv
...
```

Each file contains observations for its PRI. The training program will extract I-channel values and exclude Q-channel values.

B. PRI Analysis CSV file

Label information

Example:

```
Pri_Analysis.csv
```

It should contain columns similar to:

Pri_No	Wake_Detection
1	0xDB00
2	0x200
3	0x1FF00

Other columns such as `PRI PW`, `Tx_Angle`, `Depth`, `Velocity`, `Pri_Detection`, and `Result` can also be present.

Important: The Beam file and PRI Analysis file must correspond to the same dataset/acquisition. The program will extract the PRI number from the Beam filename. For example, `Beam_Pri_1.csv` will be matched with `Pri_No = 1`.

2. Where exactly should you put these files?

On your office laptop, create this folder structure:

```
Wake_EfficientNet_Training/
│
├── dataset/
│   ├── Beam_Files/
│   │   ├── Beam_Pri_1.csv
│   │   ├── Beam_Pri_2.csv
│   │   ├── Beam_Pri_3.csv
│   │   ├── Beam_Pri_4.csv
│   │   └── ...
│   │
```

Take your curiosity further
Scan to get the ChatGPT app

```
│   └── PRI_Analysis/
│       └── Pri_Analysis.csv
│
├── src/
├── processed/
├── checkpoints/
├── results/
│
├── train.py
├── test.py
├── predict.py
├── requirements.txt
└── README.md
```

Where to copy your files:

Your actual file	Destination
All `Beam_Pri_*.csv` files	`dataset/Beam_Files/`
`Pri_Analysis.csv`	`dataset/PRI_Analysis/`

You do not need to upload these files to Google Colab, Kaggle, or any online model-training website. We can train locally on your office laptop.

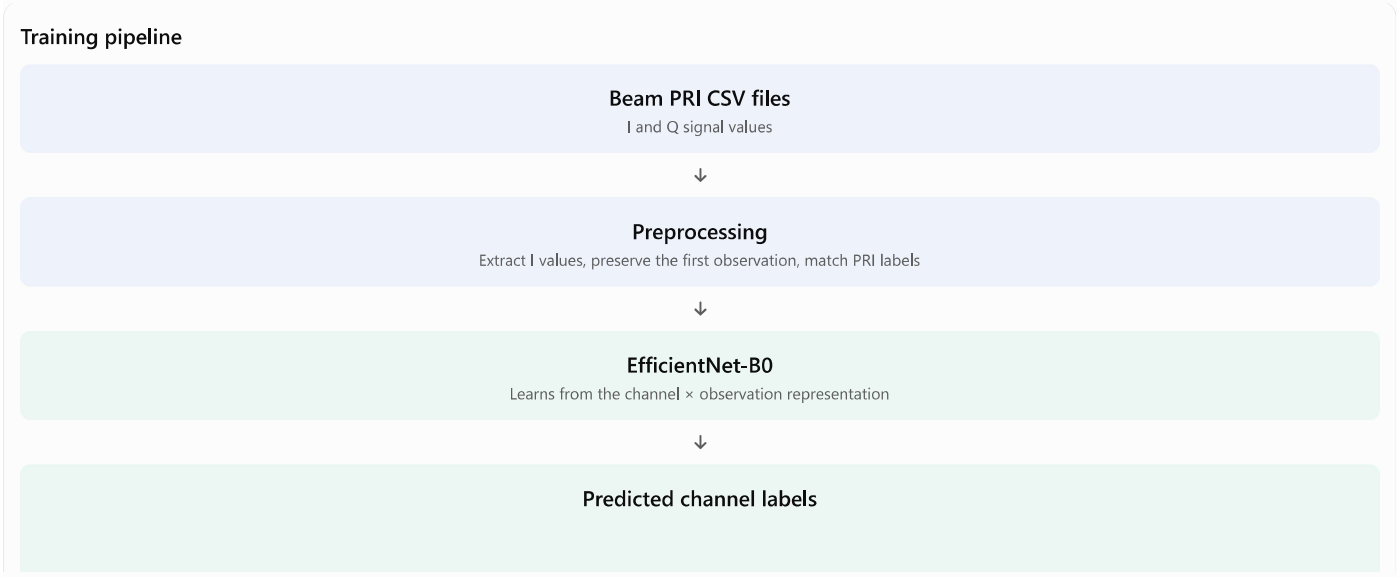
3. What will happen during training?

Let's take a simple example with three channels.

Suppose the Beam data contains:

Channel	I values	Wake label
I0	125, 130, 128, 140	YES
I1	89, 92, 85, 87	NO
I2	240, 245, 238, 250	YES

The model receives the I-channel values and learns to predict one label for each channel.



Take your curiosity further
Scan to get the ChatGPT app

Channel	Prediction
I0	YES
I1	NO
I2	YES

For EfficientNet, we will convert the channel-by-observation matrix into a consistent image-like input. The model will have a multi-label output: one YES/NO prediction per channel.

The conversion must preserve the entire observation sequence, including the first I value. If different files have different observation lengths, we must define a consistent resizing or padding method rather than silently dropping values.

One important point about your labels: We have discussed decoding `Wake_Detection` as a bitmask, with bit 0 representing I0, bit 1 representing I1, and so on. Before training, we must confirm that mapping with your guide or the dataset specification. Otherwise, the model could learn incorrect labels.

4. How to install everything on the office laptop

We will use a Python virtual environment so that packages are installed inside the project and not globally on your office laptop.

Open Command Prompt and navigate to the project folder:

```
cd path\to\Wake_EfficientNet_Training
```

Check Python:

```
py -3.12 --version
```

Create a virtual environment:

```
py -3.12 -m venv .venv
```

Activate it:

```
.venv\Scripts\activate
```

Upgrade pip:

```
python -m pip install --upgrade pip
```

Install PyTorch CPU version if you do not have a supported NVIDIA GPU:

```
python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Install the other libraries:

```
python -m pip install numpy pandas scikit-learn matplotlib seaborn pillow tqdm
```

Take your curiosity further
Scan to get the ChatGPT app

Save the installed package versions:

```
python -m pip freeze > requirements.txt
```

If the office laptop has an NVIDIA GPU, we should choose the matching CUDA installation instead of automatically installing the CPU version.

5. How will the model be trained?

After placing your data in the folders, run:

```
.venv\Scripts\activate
python train.py
```

The training program will:

1. Read the Beam PRI CSV files.
2. Read the PRI Analysis CSV file.
3. Match the PRI numbers.
4. Extract the I-channel values and remove Q-channel values.
5. Preserve the first observation from every I channel.
6. Decode the wake labels after confirming the bit mapping.
7. Create training, validation, and test datasets.
8. Train EfficientNet-B0.
9. Evaluate the model.
10. Save the best model and training results.

We should split data by recording/session wherever possible, rather than randomly splitting individual observations from the same recording. This prevents the model from being tested on data that is too similar to its training data.

6. How will you test the trained model?

There are two types of testing.

A. Testing during model development

After training, run:

```
python test.py
```

The test program should report:

Evaluation measure	What it tells us
Accuracy	How many predictions are correct
Precision	How many predicted YES labels are actually YES
Recall	How many actual YES labels were detected
F1-score	Balance between precision and recall
Confusion matrix	Correct and incorrect YES/NO predictions
Per-channel metrics	How well each individual channel performs

We should also examine false negatives carefully because missing a real wake detection may be important for your project.

Take your curiosity further
Scan to get the ChatGPT app

B. Testing with a new Beam PRI file

After the model is trained, give it a Beam file that was not used for training.

For example:

```
New_Beam_Files/
    Beam_Pri_101.csv
    Beam_Pri_102.csv
```

Run:

```
python predict.py
```

The model will produce a result similar to:

Channel	Predicted wake	Confidence
I0	YES	94%
I1	NO	12%
I2	YES	87%
I3	NO	21%

These percentages are model scores, not guaranteed probabilities of real-world detection. The decision threshold should be selected using validation data.

Important: The test dataset must be kept separate from the training dataset. Do not use the same Beam files for both and then report the result as independent test performance.

7. What files will be created after training?

After training, your project should contain something similar to:

```
Wake_EfficientNet_Training/
│
├── checkpoints/
│   ├── best_efficientnet_b0.pth
│   └── last_efficientnet_b0.pth
│
├── results/
│   ├── training_history.csv
│   ├── accuracy.png
│   ├── loss.png
│   ├── confusion_matrix.png
│   ├── channel_metrics.csv
│   └── test_predictions.csv
│
├── processed/
│   └── preprocessing_config.json
│
└── requirements.txt
```

Take your curiosity further
Scan to get the ChatGPT app

The most important files to transfer to your GUI laptop are:

- `best_efficientnet_b0.pth` — trained model weights.
- `preprocessing_config.json` — the preprocessing settings used during training.
- Model configuration — channel count, input size, architecture and threshold settings.

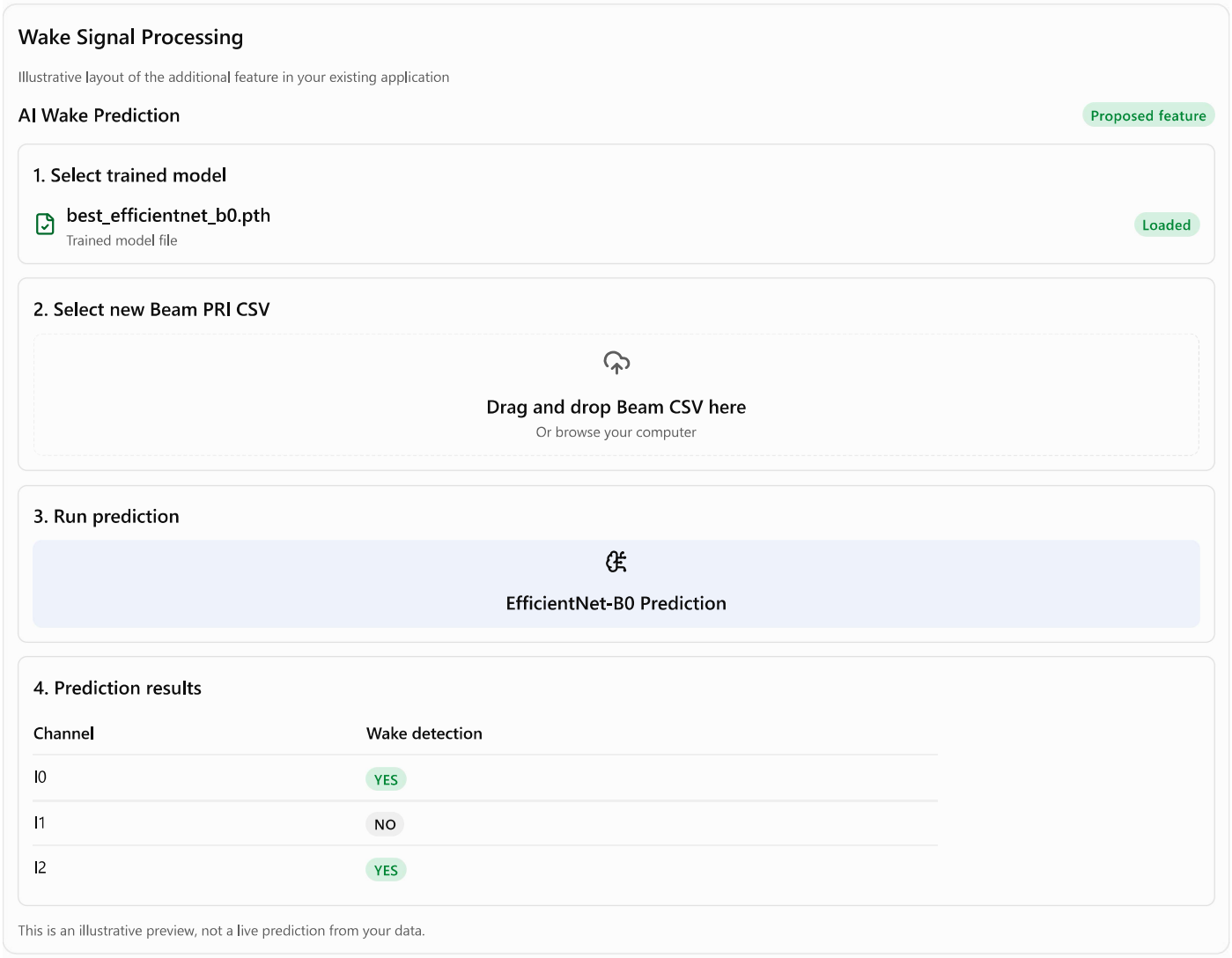
The exact architecture definition and model code must also be included in the GUI application. A `.pth` file alone is not a complete runnable model.

8. Do you need a new website or GUI?

No new GUI required

We can continue with your existing PySide6 GUI.

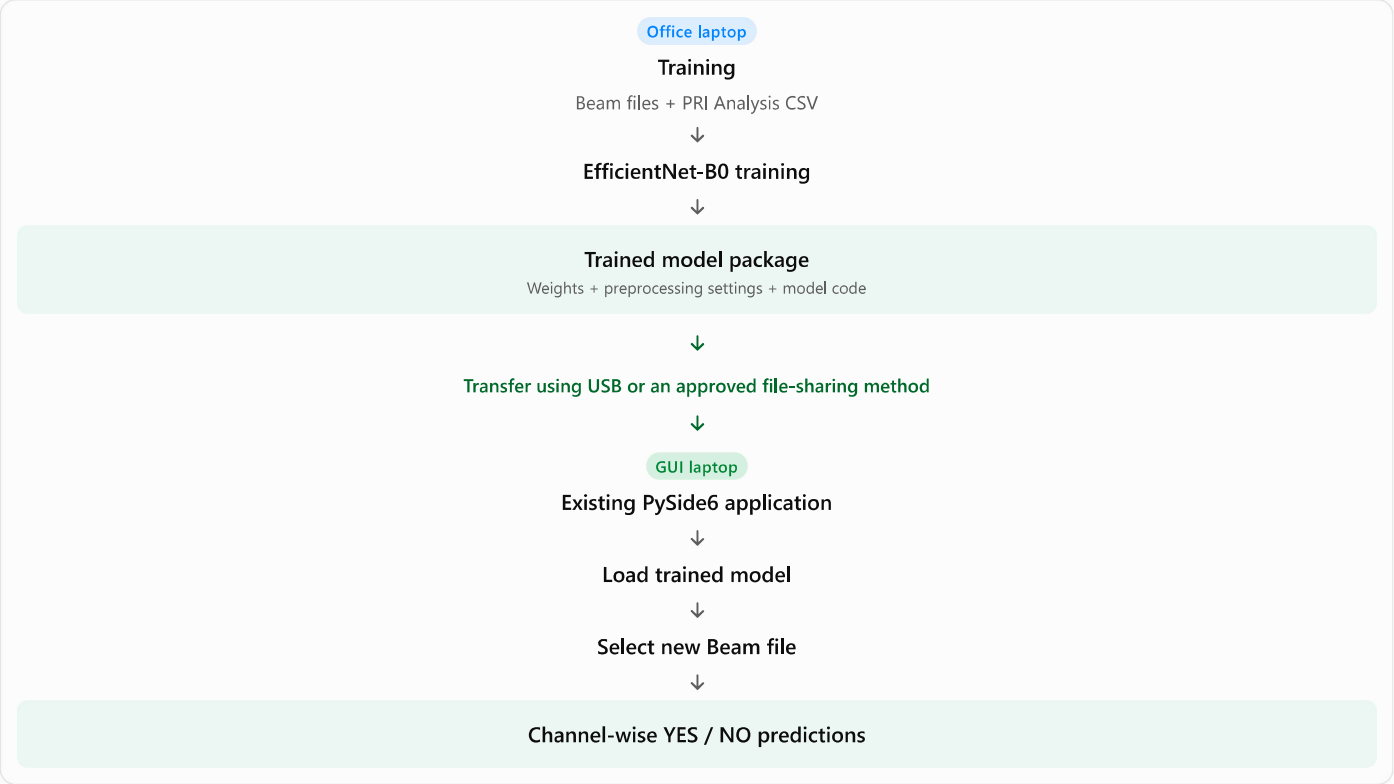
We only need to add a new feature called **AI Wake Prediction**.



Your existing GUI can continue handling file selection, preprocessing and saving. We will add model loading, a prediction button and a results table. You do not need a website, server, or a second GUI.

Take your curiosity further
Scan to get the ChatGPT app

9. How will the office laptop and GUI laptop work together?



You can perform predictions without an internet connection once the necessary packages and model files are installed. If we package the updated GUI with PyInstaller, we can also make it easier to run on another Windows computer.

10. What I need to prepare the complete training project correctly

Before I create the final EfficientNet training project with `train.py`, `test.py`, `predict.py`, preprocessing code, and Windows batch files, I need to confirm a few details.

Office laptop and dataset details

1. Does the office laptop have an NVIDIA GPU?

- ○ Yes
- ○ No, CPU only
- ○ I don't know

2. How much RAM does it have?

- ○ 8 GB
- ○ 16 GB
- ○ 32 GB
- ○ 64 GB or more
- ○ I don't know

Take your curiosity further
Scan to get the ChatGPT app

3. Approximately how many Beam PRI CSV files do you have?

- Below 100
- 100–500
- 500–1000
- More than 1000

4. Are the number of observations in each Beam file the same?

- Same number
- Different numbers
- I don't know

5. How are the Beam CSV columns arranged?

- I0, Q0, I1, Q1, etc.
- Named columns such as I0, I1, I2, with Q columns separately
- I don't know / generic column names

6. How many PRI Analysis files do you have?

- One common PRI Analysis CSV for all Beam files
- Multiple PRI Analysis CSV files
- Not sure

Submit details

One more important thing: If possible, provide one sample `Beam_Pri_*.csv` and the corresponding `Pri_Analysis.csv` from the office dataset. You can remove or mask any sensitive information. Those two sample files will let me verify the column arrangement, first-value preservation, PRI matching and hexadecimal label decoding before preparing the final training code.

My recommendation is to keep the existing GUI and develop the model-training project separately. Once EfficientNet is trained and tested properly, we will integrate it into the same GUI rather than rebuilding the application.

You

while model training is it needed to give the outputs also or only beam and pri files only

ChatGPT

For EfficientNet model training, **you mainly need the Beam CSV files and PRI Analysis CSV file. You do not need to provide the output files generated by your existing GUI.**

However, there is one important condition: the PRI Analysis file must contain the correct wake labels.

Take your curiosity further
Scan to get the ChatGPT app

1. Files required for training

File	Required?	Purpose
Beam PRI CSV files	Yes	Input I-channel signal values
PRI Analysis CSV	Yes	Wake_Detection labels
Previously generated GUI output CSV	No	Not needed if labels can be decoded from PRI Analysis
Manually labelled output files	Only if needed	Needed if the PRI Analysis file does not provide valid channel-wise labels

2. How will the training program use them?

For example:

Beam_Pri_1.csv

Contains the I/Q signal values for different channels.

Pri_Analysis.csv

Pri_No	Wake_Detection
1	0xDB00
2	0x200
3	0x1FF00

The program will:

1. Read `Beam_Pri_1.csv`.
2. Find PRI number 1 in `Pri_Analysis.csv`.
3. Read its `Wake_Detection` value.
4. Decode the hexadecimal value into channel-wise YES/NO labels, according to the verified bit mapping.
5. Use the I-channel values as input and the decoded labels as training targets.

3. When would additional output files be required?

If your PRI Analysis file contains only detection information but does not provide reliable channel-wise YES/NO labels, we need a separate correctly labelled output dataset.

My recommendation: Initially, provide only:

- All Beam PRI CSV files.
- The corresponding PRI Analysis CSV file.

Also provide one sample of each so I can verify whether `Wake_Detection` is sufficient to generate the training labels.

Remember: The model cannot learn supervised YES/NO detection from signal values alone. It needs correct labels during training.

Take your curiosity further
Scan to get the ChatGPT app